MACSE502 Programming for Data Science

Sample Questions: Model Answers

Answers to the 18 higher-order-thinking (HOT) questions of the *Sample Questions* folder: 3 on Python data structures, APIs and NLP, 5 on Python libraries, 5 on Scala, 5 on Spark, Breeze and Saddle. Each answer replies to every bullet of its question and comes with a program that was run on the dataset printed in the question.

Question	Topic	Main tools	Code (Question_Bank/code)
M1-Q1	Library borrowing system	dict, list, set, deque, heapq, exceptions, text file	sa_m1_q1.py
M1-Q2	Weather monitoring with REST APIs	requests, JSON, retries, exception classes, files	sa_m1_q2.py, sa_m1_q2_stub.py
M1-Q3	Student support chatbot	NLTK, ChatterBot, classes, Counter, deque	sa_m1_q3.py
M2-Q1	Student performance: missing values, statistics, risk	pandas, scikit-learn imputer, Matplotlib	sa_m2_q1.py
M2-Q2	Customer purchases: filtering, text data, insights	pandas str methods, crosstab	sa_m2_q2.py
M2-Q3	Web data mining and merging	Beautiful Soup, Scrapy, pandas merge	sa_m2_q3.py, sa_m2_q3_scrapy.py
M2-Q4	House price prediction	NumPy, SciPy, scikit-learn	sa_m2_q4.py
M2-Q5	Smart city dashboard	Matplotlib, Seaborn, Plotly, Bokeh, Folium	sa_m2_q5.py
M3-Q1	Course registration	classes, constructors, List, overloading, inheritance	sa_m3_q1.scala
M3-Q2	Library management	abstract class, field overriding, polymorphism, final	sa_m3_q2.scala
M3-Q3	Shopping cart	Map, map / filter / reduce, imperative vs functional	sa_m3_q3.scala
M3-Q4	Employee management	case class, companion, access modifiers, singleton	sa_m3_q4.scala
M3-Q5	Transport monitoring	traits, mixins, Array, REPL	sa_m3_q5.scala, sa_m3_q5_repl.txt
M4-Q1	Student analytics at scale	Spark architecture, Breeze, MLlib	sa_m4_q1.scala
M4-Q2	Healthcare data	Saddle Series / Frame, Spark	sa_m4_q2_saddle.scala, sa_m4_q2_spark.scala
M4-Q3	E-commerce recommender	exception handling, fault tolerance, Spark	sa_m4_q3.scala
M4-Q4	Social-media big data	driver, executors, scheduling, metrics	sa_m4_q4.scala
M4-Q5	Financial risk	Breeze, Saddle, Spark	sa_m4_q5_saddle_breeze.scala, sa_m4_q5_spark.scala

Watch out. These are model answers written for revision. I do not know the marking scheme of the examiner, so no marks are given, and the points a particular examiner expects may differ. Every program and every number in the printed outputs was produced by really running it.

Contents

How to read and run this file **3**

Module 1: Python data structures, REST APIs and NLP **4**

M1-Q1 Library Borrowing System: lists, dictionaries, sets, queues and priority queues 4

M1-Q2 Weather Monitoring and Analysis using REST APIs 9

M1-Q3 Intelligent Student Support Chatbot using NLTK and ChatterBot 14

Module 2: Python libraries for data science **18**

M2-Q1 Student Performance Analysis using Pandas 18

M2-Q2 Customer Purchase Pattern Analysis using DataFrame Operations 22

M2-Q3 Web Data Mining and Integration (Scrapy, Beautiful Soup, merging) 25

M2-Q4 Predicting House Prices using Machine Learning 30

M2-Q5 Smart City Analytics Dashboard 33

Module 3: Scala data structures and object-oriented programming **37**

M3-Q1 Student Course Registration System 37

M3-Q2 Smart Library Management System 41

M3-Q3 Online Shopping Cart Analysis 43

M3-Q4 Employee Management using Case Classes and Companion Objects 45

M3-Q5 Smart Transportation Monitoring System 47

Module 4: Scala libraries and Spark basics **50**

M4-Q1 Large-Scale Student Performance Analytics using Spark and Breeze 50

M4-Q2 Smart Healthcare Data Processing with Saddle and Spark 54

M4-Q3 E-Commerce Recommendation System with Exception Handling and Spark 58

M4-Q4 Social Media Sentiment Analysis using Spark Big Data Architecture 62

M4-Q5 Financial Risk Analysis using Breeze, Saddle and Spark 66

Notes and limits **70**

How to read and run this file

How each answer is built

1. The question as given (the screenshot), so the dataset and the bullets are in front of you.
2. **Answer, point by point**: one block per bullet, in the order asked, with the tool named in the first sentence and the reason ("why this and not that") stated, because HOT questions reward justification.
3. **Code, run on the dataset of the question**, followed by its real output. Numbers quoted in the text come from that output.
4. Where something had to be made up (invented posts, map coordinates, stand-in web pages) the answer says so.

A five-step recipe for any scenario question

1. **Restate the goal** in one line (what decision must the answer support?).
2. **Answer every bullet** in the order asked, a short paragraph or a small table each. Use the library names from the question.
3. **Show the central code** in a few lines: load, clean, compute, show. Do not print a whole program unless asked.
4. **Justify**: compare at least two options for every important choice (median or mean, List or Array, Spark or pandas) and say why one wins here.
5. **State the limits**: small sample, assumptions made, and one next step with more data.

How to run the programs

Python programs: `python sa_m2_q1.py` in a Python 3.10 environment with pandas, NumPy, SciPy, scikit-learn, Matplotlib, Seaborn, Plotly, Bokeh, Folium and Beautiful Soup. Extra needs: `sa_m1_q2.py` needs `requests` and talks to the stand-in service, which it imports and starts from `sa_m1_q2_stub.py`; `sa_m1_q3.py` needs NLTK (with its `punkt` and `stopwords` data) and ChatterBot, which is best kept in its own virtual environment because it installs many dependencies (spaCy among them) that can clash with the other libraries; `sa_m2_q3_scrapy.py` needs `pip install scrapy`. Scala programs: `scala-cli run sa_m3_q1.scala` (Scala 2.13.12, Java 17). The `//> using dep` lines at the top of a file download the libraries.

The Spark programs start Spark in local mode (`master("local[2]")`, one machine, no cluster). On Java 17 Spark 3.5 needs the following line in each Spark file, which the listings abbreviate:

The javaOpt line of the Spark programs

```
//> using javaOpt --add-opens=java.base/java.lang=ALL-UNNAMED --add-opens=java.base/java.lang.invoke=ALL-UNNAMED
    --add-opens=java.base/java.lang.reflect=ALL-UNNAMED --add-opens=java.base/java.io=ALL-UNNAMED --add-
    opens=java.base/java.net=ALL-UNNAMED --add-opens=java.base/java.nio=ALL-UNNAMED --add-
    opens=java.base/java.util=ALL-UNNAMED --add-opens=java.base/java.util.concurrent=ALL-UNNAMED --add-
    opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED --add-opens=java.base/sun.nio.ch=ALL-UNNAMED --add-
    opens=java.base/sun.nio.cs=ALL-UNNAMED --add-opens=java.base/sun.security.action=ALL-UNNAMED --add-
    opens=java.base/sun.util.calendar=ALL-UNNAMED
```

Tip. Three small programs are meant to fail (`sa_m3_q2_final_error.scala`, `sa_m3_q2_final_class_error.scala`, `sa_m3_q4_private_error.scala`); their compiler messages are quoted in the answers. `run_sa_extras.py` produces those messages and the REPL transcript.

Module 1: Python data structures, REST APIs and NLP

3 sample questions with model answers.

M1-Q1 Library Borrowing System: lists, dictionaries, sets, queues and priority queues

Books Dataset

Book ID	Title	Author	Category	Available Copies
B101	Python Programming	John Smith	Programming	2
B102	Data Structures	Robert Brown	Computer Science	0
B103	Machine Learning Basics	Alice Green	AI	1
B104	Database Systems	David Lee	Database	0
B105	Web Development	Emma Wilson	Programming	3

The question as given, part 1 of 5 (Sample Questions/6260159273853719032.jpg)

Members Dataset

Member ID	Member Name	Membership Type
M001	Ravi Kumar	Premium
M002	Priya Singh	Standard
M003	Arjun Das	Premium
M004	Meena Rao	Standard

The question as given, part 2 of 5 (Sample Questions/6260159273853719033.jpg)

Borrow Request Dataset

Request ID	Member ID	Book ID	Request Date
R001	M001	B102	10-10-2026
R002	M002	B103	11-10-2026
R003	M003	B104	11-10-2026
R004	M004	B101	12-10-2026
R005	M001	B104	12-10-2026

The question as given, part 3 of 5 (Sample Questions/6260159273853719035.jpg)

Waiting List Dataset

Queue Position	Member ID	Book ID
1	M001	B102
2	M003	B104
3	M001	B104

The question as given, part 4 of 5 (Sample Questions/6260159273853719036.jpg)

Tasks Based on the Dataset

1. Store the **Books Dataset** using a suitable Python data structure.
2. Maintain the **Waiting List Dataset** using a Queue.
3. Identify books that are currently unavailable.
4. Display all requests made by Premium members.
5. Handle the exception when a user requests a non-existing book (e.g., `B110`).
6. Record all successful borrow transactions in a text file.
7. Suggest where Lists, Dictionaries, Sets, Queues, and Priority Queues can be applied in the system.

The question as given, part 5 of 5 (Sample Questions/6260159273853719037.jpg)

Note. This question arrives as five screenshots (Books, Members, Borrow Request and Waiting List datasets, then the seven tasks). The picture numbered ...034 that would hold the opening scenario text is not in the folder, so the answer works from the datasets and the seven tasks. The waiting list is consistent with the borrow requests: it holds exactly the requests for books with 0 copies (B102 and B104), in order.

Answer, task by task

1. Store the Books dataset using a suitable Python data structure

A **dictionary of dictionaries** keyed by BookID: `books["B103"]["title"]`. The BookID is the natural key, a lookup by key takes the same time for 5 or 5 million books, and the inner dictionary gives every field a name.

Structure	Fits?	Reason
dict of dicts (chosen)	yes	O(1) lookup by BookID, readable field names, easy to update `copies`
list of dicts	partly	keeps order and is easy to loop over, but finding B103 means scanning the list (O(n))
list of tuples / namedtuple	partly	compact; tuples are immutable, so a changing `copies` needs a new tuple
pandas DataFrame	for analysis	best once you filter and group whole columns (Module 2); heavy for a few look-ups

The other datasets follow the same rule: Members in a dictionary (`MemberID -> (name, type)`), the Borrow Requests in a **list of tuples** (an ordered log in which the same member may appear many times), the Waiting List in a queue.

2. Maintain the Waiting List using a Queue

- A queue is first in, first out. Use `collections.deque`: `append(x)` joins at the back, `popleft()` serves the front, both in O(1). `list.pop(0)` shifts every element and is O(n); `queue.Queue` is meant for threads.
- The output shows the dataset as a queue, a new request joining at the back, and `popleft()` serving ('M001', 'B102') first when a copy of B102 returns.
- **Design point:** when a copy of B104 returns, the member to serve is the first one waiting for **B104** (M003), not the front of the global queue (M001 waits for B102). A dictionary with one deque per book, `{book: deque}`, gives exactly that; the code prints it.

3. Identify books that are currently unavailable

A list comprehension over the dictionary: `[b for b, d in books.items() if d["copies"] == 0]` returns **B102 (Data Structures)** and **B104 (Database Systems)**.

4. Display all requests made by Premium members

Build a **set** of Premium member ids (`{m for m, (_, kind) in members.items() if kind == "Premium"}`), because "is this member in the set?" is a constant-time test, then filter the request list with it. Result: **R001** (Ravi Kumar, Data Structures), **R003** (Arjun Das, Database Systems), **R005** (Ravi Kumar, Database Systems).

5. Handle the exception when a user requests a non-existing book (B110)

- Define your own exception, `class BookNotFoundError(Exception)`, and turn the low-level `KeyError` into it inside `find_book`: `except KeyError: raise BookNotFoundError(...) from None`. The caller then catches one meaningful error.
- The program prints "book B110 does not exist in the catalogue" and continues: in the transaction loop request R006 for B110 is skipped, not fatal.
- Running out of copies is **not** an exception but a normal outcome: that request goes to the waiting list.

6. Record all successful borrow transactions in a text file

- A request succeeds if a copy is in stock when it is processed. The loop reduces the stock by one and writes a line with `with open(path, "w") as f: f.write(...)` (use mode `"a"` to append to an existing log).
- Result in the file: **R002** (M002, B103) and **R004** (M004, B101). The other requests had no copy and went to the waiting list, which reproduces the Waiting List dataset exactly (the program prints `True`).
- `with` closes the file even if an error occurs; the file is read back to prove what was written.

7. Where can Lists, Dictionaries, Sets, Queues and Priority Queues be applied?

Structure	Use in the library system
List	ordered log of requests and transactions (append, loop, slice), results of searches
Dictionary	books by BookID, members by MemberID, copies in stock, one waiting queue per book, counts of borrows per category
Set	Premium member ids (fast membership test), the distinct categories, members who have ever borrowed, members who requested but never borrowed (set difference)
Queue (deque)	the waiting list: first come, first served
Priority queue (`heapq`)	serve Premium members before Standard members for the same book, ties broken by arrival order

The last row is run in the code as an illustration: if Priya (Standard) had asked for B104 first, the priority queue still serves Arjun Das and Ravi Kumar (Premium) before her. The entries are tuples `(rank, arrival counter, member, book)`; the counter keeps arrival order among equal ranks.

Watch out. A priority queue changes fairness: Standard members can wait much longer. State the rule explicitly (for example, Premium first but nobody waits more than 7 days).

Code, run on the datasets of the question

Code: code/sa_m1_q1.py

```
"""Sample answer M1-Q1: library borrowing system with a dictionary, list, set, queue, priority queue, exceptions and
     a text file."""
import heapq
import itertools
import os
import tempfile
from collections import deque

# The datasets of the question, each in the structure that suits it
books = {                                                  # dictionary of dictionaries: BookID -> details
    "B101": {"title": "Python Programming", "author": "John Smith", "category": "Programming", "copies": 2},
    "B102": {"title": "Data Structures", "author": "Robert Brown", "category": "Computer Science", "copies": 0},
    "B103": {"title": "Machine Learning Basics", "author": "Alice Green", "category": "AI", "copies": 1},
    "B104": {"title": "Database Systems", "author": "David Lee", "category": "Database", "copies": 0},
    "B105": {"title": "Web Development", "author": "Emma Wilson", "category": "Programming", "copies": 3}}
members = {"M001": ("Ravi Kumar", "Premium"), "M002": ("Priya Singh", "Standard"),
           "M003": ("Arjun Das", "Premium"), "M004": ("Meena Rao", "Standard")}      # MemberID -> (name, type)
requests = [("R001", "M001", "B102", "10-10-2026"), ("R002", "M002", "B103", "11-10-2026"), ("R003", "M003", "B104",
     "11-10-2026"),
            ("R004", "M004", "B101", "12-10-2026"), ("R005", "M001", "B104", "12-10-2026")]    # list of tuples:
                ordered, repeats allowed
waiting = deque([("M001", "B102"), ("M003", "B104"), ("M001", "B104")])                 # queue: position = order
    in the deque

print("--- 1. the Books dataset in a dictionary keyed by BookID ---")
print(len(books), "books | title of B103:", books["B103"]["title"], "| copies of B105:", books["B105"]["copies"])
print("categories (a set removes repeats):", sorted({d["category"] for d in books.values()}))

print("\n--- 2. the waiting list is a FIFO queue (collections.deque) ---")
initial_waiting = list(waiting)
print("front to back:", list(waiting))
waiting.append(("M002", "B102"))                           # a new request joins at the back
print("a copy of B102 comes back -> serve:", waiting.popleft(), "| still waiting:", list(waiting))
waiting = deque(initial_waiting)                           # restore for the next steps
per_book = {}                                              # a dictionary of queues: one FIFO queue per book
for member, book in initial_waiting:
    per_book.setdefault(book, deque()).append(member)
print("one queue per book:", {b: list(q) for b, q in per_book.items()})
print("a copy of B104 comes back -> serve:", per_book["B104"].popleft(), "(the front of the global queue waits for
    B102, not B104)")

print("\n--- 3. books that are currently unavailable ---")
unavailable = [b for b, d in books.items() if d["copies"] == 0]
print(unavailable, [books[b]["title"] for b in unavailable])

print("\n--- 4. requests made by Premium members ---")
premium_ids = {m for m, (_, kind) in members.items() if kind == "Premium"}      # a set: fast "is this member
    Premium?"
for rid, mid, bid, date in (r for r in requests if r[1] in premium_ids):
    print(rid, members[mid][0], "asked for", books[bid]["title"], "on", date)


print("\n--- 5. exception for a non-existing book ---")
class BookNotFoundError(Exception):
    pass


def find_book(book_id):
    try:
        return books[book_id]
    except KeyError:
        raise BookNotFoundError(f"book {book_id} does not exist in the catalogue") from None


for bid in ("B101", "B110"):
    try:
        print(bid, "->", find_book(bid)["title"])
    except BookNotFoundError as e:
        print("error:", e)

print("\n--- 6. successful borrow transactions are written to a text file ---")
log_path = os.path.join(tempfile.mkdtemp(), "borrow_log.txt")
stock = {b: d["copies"] for b, d in books.items()}          # working copy of the stock
unserved = []
with open(log_path, "w", encoding="utf-8") as f:
    for rid, mid, bid, date in requests + [("R006", "M004", "B110", "13-10-2026")]:      # R006 asks for a book that
         does not exist
        try:
            find_book(bid)
        except BookNotFoundError as e:
            print("skipped", rid, "->", e)
            continue
        if stock[bid] > 0:
            stock[bid] -= 1
            f.write(f"{rid},{mid},{bid},{date},BORROWED\n")
```

```
        else:
            unserved.append((mid, bid))
with open(log_path, encoding="utf-8") as f:
    print(f.read().strip())
print("stock afterwards:", stock)
print("requests that had to wait equal the waiting-list dataset:", unserved == initial_waiting)

print("\n--- 7. a priority queue: Premium members are served before Standard members ---")
order, pq = itertools.count(), []                          # the counter keeps arrival order among equals


def enqueue(member, book):
    rank = 0 if members[member][1] == "Premium" else 1
    heapq.heappush(pq, (rank, next(order), member, book))


for m, b in [("M002", "B104"), ("M003", "B104"), ("M001", "B104")]:       # illustration: Priya (Standard) asked
    first
    enqueue(m, b)
print("served in this order:", [members[heapq.heappop(pq)[2]][0] for _ in range(3)])
```

Real output

```
--- 1. the Books dataset in a dictionary keyed by BookID ---
5 books | title of B103: Machine Learning Basics | copies of B105: 3
categories (a set removes repeats): ['AI', 'Computer Science', 'Database', 'Programming']

--- 2. the waiting list is a FIFO queue (collections.deque) ---
front to back: [('M001', 'B102'), ('M003', 'B104'), ('M001', 'B104')]
a copy of B102 comes back -> serve: ('M001', 'B102') | still waiting: [('M003', 'B104'), ('M001', 'B104'), ('M002',
    'B102')]
one queue per book: {'B102': ['M001'], 'B104': ['M003', 'M001']}
a copy of B104 comes back -> serve: M003 (the front of the global queue waits for B102, not B104)

--- 3. books that are currently unavailable ---
['B102', 'B104'] ['Data Structures', 'Database Systems']

--- 4. requests made by Premium members ---
R001 Ravi Kumar asked for Data Structures on 10-10-2026
R003 Arjun Das asked for Database Systems on 11-10-2026
R005 Ravi Kumar asked for Database Systems on 12-10-2026

--- 5. exception for a non-existing book ---
B101 -> Python Programming
error: book B110 does not exist in the catalogue

--- 6. successful borrow transactions are written to a text file ---
skipped R006 -> book B110 does not exist in the catalogue
R002,M002,B103,11-10-2026,BORROWED
R004,M004,B101,12-10-2026,BORROWED
stock afterwards: {'B101': 1, 'B102': 0, 'B103': 0, 'B104': 0, 'B105': 3}
requests that had to wait equal the waiting-list dataset: True

--- 7. a priority queue: Premium members are served before Standard members ---
served in this order: ['Arjun Das', 'Ravi Kumar', 'Priya Singh']
```

M1-Q2 Weather Monitoring and Analysis using REST APIs

2. Weather Monitoring and Analysis using REST APIs

A weather analytics application collects live weather information from multiple cities using REST APIs. The application should:

- Send HTTP requests to fetch weather data.
- Process and store the response data.
- Handle network failures and invalid API responses.
- Generate a summary report and save it to a file.

Question: Explain the complete workflow of the application. Discuss how APIs, exception handling, dictionaries, and file handling work together to ensure robust and reliable data collection and analysis.

The question as given (Sample Questions/6260159273853719038.jpg)

Answer

The complete workflow

Step	What happens	Python tools
1 Configure	list of cities, base URL, timeout and retry limit; the API key comes from an environment variable, never from the source code	`os.environ`, constants
2 Request	one HTTP GET per city with query parameters, over one reused connection and with a timeout	`requests.Session`, `session.get(url, params=..., timeout=...)`
3 Check the answer	status code, JSON parsing, required fields, types and plausible ranges	`resp.status_code`, `raise_for_status`, `resp.json()`, `validate()`
4 Process and store	valid readings go into one dictionary (city -> reading), failures into another (city -> reason)	`dict`
5 Handle failures	transient problems are retried with a growing pause; invalid answers are recorded, not retried; one bad city never stops the others	`try / except`, custom exceptions, `time.sleep`
6 Summarise	counts, average, hottest and coldest city, list of problems	`statistics`, `max`, `min`
7 Save	summary as text, readings as JSON	`with open(...)`, `json.dump`

How APIs, exception handling, dictionaries and file handling work together

- The **API** is the source of the data but cannot be trusted: it may be slow, down, or send nonsense. Everything that crosses this boundary is checked.
- **Exception handling** is the safety net around each external step. A small hierarchy (`WeatherError` with `NetworkError` and `InvalidResponse` below it) lets the program decide: retry transient problems, only record invalid ones. A broad `except Exception` would hide programming errors, so the handlers are specific.
- **Dictionaries** hold the result in a form that is easy to use: `results[city]` for readings, `failures[city]` for reasons. Together they account for every city requested (4 + 5 = 9).
- **File handling** makes the outcome durable and auditable: `summary.txt` for people, `readings.json` for other programs. `with` guarantees the files are closed even when something fails.

What makes it robust and reliable

Problem	In the test run	Handled by
Server error HTTP 503 (busy)	Bengaluru fails once, then works	retry with a pause: "ok after 2 attempt(s)"
Slow server	Slowtown answers after 2.5 s, timeout is 1 s	`timeout=` raises `ReadTimeout`; after the retries it is recorded as a network problem
Client error HTTP 404	Atlantis	not retried (our request is wrong): "request rejected: HTTP 404"
Answer is not JSON	Brokenville returns an HTML page	`resp.json()` fails, turned into `InvalidResponse`
Field missing	Nulltown has no `temp_c`	`validate()` raises `InvalidResponse`
Impossible value	Hotville says 999 C	range check in `validate()`
Connection refused or lost	same family as the timeout	`requests.ConnectionError` is retried like a timeout

Tip. Never call `requests.get` without `timeout=`: by default it can wait forever. Retry only what can change on a second try (timeouts, 5xx), never 4xx errors or invalid data.

Note. The weather service is a small local stand-in server (`sa_m1_q2_stub.py`) that misbehaves on purpose for the cities above, so that every failure can be reproduced offline. With a real provider only `BASE_URL` and the API key change.

Code, run against the stand-in service

Code: code/sa_m1_q2.py (the client)

```python
"""Sample answer M1-Q2: a weather collector that survives network failures and invalid API answers.
The service is a local stand-in (sa_m1_q2_stub.py); with a real service only BASE_URL (and an API key) changes."""
import json
import os
import statistics
import tempfile
import time

import requests

from sa_m1_q2_stub import start_stub

BASE_URL = start_stub()                      # real life: "https://api.<weather-provider>/v1" and the API key in
    params or headers


class WeatherError(Exception):
    """Base class: anything that went wrong while collecting weather data."""


class NetworkError(WeatherError):
    """Transient problem (timeout, connection lost, HTTP 5xx): worth retrying."""


class InvalidResponse(WeatherError):
    """The server answered, but the answer cannot be used."""


REQUIRED = {"city": str, "temp_c": (int, float), "humidity": (int, float)}


def validate(data):
    if not isinstance(data, dict):
        raise InvalidResponse("the answer is not a JSON object")
    for key, kind in REQUIRED.items():
        if key not in data:
            raise InvalidResponse(f"missing field '{key}'")
        if not isinstance(data[key], kind):
            raise InvalidResponse(f"field '{key}' has the wrong type")
    if not -60 <= data["temp_c"] <= 60:
        raise InvalidResponse(f"temperature {data['temp_c']} C is impossible")
    return data


def fetch(session, city, retries=2, timeout=1.0):
    """GET one city. Returns (data, number of attempts). Retries only transient problems, with a growing pause."""
    last = None
    for attempt in range(1, retries + 2):
        try:
            resp = session.get(f"{BASE_URL}/weather", params={"city": city}, timeout=timeout)
            if resp.status_code >= 500:
                raise NetworkError(f"server error {resp.status_code}")
            resp.raise_for_status()                          # 4xx: our request is wrong, retrying would not help
            try:
                data = resp.json()
            except ValueError:
                raise InvalidResponse("the answer is not JSON") from None
            return validate(data), attempt
        except (requests.ConnectionError, requests.Timeout) as e:
            last = NetworkError(type(e).__name__)
        except NetworkError as e:
            last = e
        except requests.HTTPError as e:
            raise WeatherError(f"HTTP {e.response.status_code}") from e
        time.sleep(0.2 * attempt)                            # back-off before the next attempt
    raise last


cities = ["Chennai", "Mumbai", "Delhi", "Bengaluru", "Atlantis", "Brokenville", "Nulltown", "Hotville", "Slowtown"]
results, failures = {}, {}                                   # dictionaries: city -> reading, city -> reason
with requests.Session() as session:
    for city in cities:
        try:
            data, attempts = fetch(session, city)
            results[city] = data
            print(f"{city:<12} ok after {attempts} attempt(s): {data['temp_c']} C, humidity {data['humidity']}%")
        except InvalidResponse as e:
            failures[city] = f"invalid response: {e}"
        except NetworkError as e:
            failures[city] = f"network problem: {e}"
        except WeatherError as e:
            failures[city] = f"request rejected: {e}"
        if city in failures:
            print(f"{city:<12} FAILED, {failures[city]}")
```

```
temps = {c: d["temp_c"] for c, d in results.items()}
lines = [f"Cities requested : {len(cities)}", f"Successful       : {len(results)}", f"Failed           :
    {len(failures)}",
         f"Average temp     : {statistics.mean(temps.values()):.1f} C",
         f"Hottest / coldest: {max(temps, key=temps.get)} {max(temps.values())} C / {min(temps, key=temps.get)}
             {min(temps.values())} C",
         f"Average humidity : {statistics.mean(d['humidity'] for d in results.values()):.1f} %", "Problems:"]
lines += [f"  {c}: {why}" for c, why in failures.items()]

folder = tempfile.mkdtemp(prefix="weather_")
with open(os.path.join(folder, "summary.txt"), "w", encoding="utf-8") as f:
    f.write("\n".join(lines) + "\n")
with open(os.path.join(folder, "readings.json"), "w", encoding="utf-8") as f:
    json.dump({"readings": results, "failures": failures}, f, indent=2)
print("\n--- summary.txt, read back from the file ---")
with open(os.path.join(folder, "summary.txt"), encoding="utf-8") as f:
    print(f.read().strip())
print("files written:", sorted(os.listdir(folder)))
```

Code: code/sa_m1_q2_stub.py (the stand-in service)

```
"""A tiny weather service that stands in for a real REST API, so that sa_m1_q2.py can be tested offline.
It misbehaves on purpose for some cities: one error, one slow answer, one flaky answer, three kinds of invalid
    answer."""
import json
import threading
import time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import parse_qs, urlparse

DATA = {"Chennai": (33.5, 70, "Sunny"), "Mumbai": (31.0, 78, "Humid"), "Delhi": (24.5, 40, "Haze"), "Bengaluru":
    (26.0, 60, "Cloudy")}
_hits = {}


class Handler(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass

    def _send(self, code, body, ctype="application/json"):
        raw = body.encode("utf-8")
        try:
            self.send_response(code)
            self.send_header("Content-Type", ctype)
            self.send_header("Content-Length", str(len(raw)))
            self.end_headers()
            self.wfile.write(raw)
        except OSError:                                 # the client gave up (timeout): nothing to do
            pass

    def do_GET(self):
        url = urlparse(self.path)
        city = parse_qs(url.query).get("city", [""])[0]
        if url.path != "/weather":
            return self._send(404, "{}")
        _hits[city] = _hits.get(city, 0) + 1
        if city == "Bengaluru" and _hits[city] == 1:                        # flaky: the first call fails, the retry
            works
            return self._send(503, json.dumps({"error": "busy"}))
        if city == "Slowtown":
            time.sleep(2.5)
            return self._send(200, json.dumps({"city": city, "temp_c": 30, "humidity": 50}))
        if city == "Brokenville":
            return self._send(200, "<html>maintenance page</html>", "text/html")
        if city == "Nulltown":
            return self._send(200, json.dumps({"city": city, "humidity": 50}))      # temperature missing
        if city == "Hotville":
            return self._send(200, json.dumps({"city": city, "temp_c": 999, "humidity": 10}))
        if city in DATA:
            t, h, c = DATA[city]
            return self._send(200, json.dumps({"city": city, "temp_c": t, "humidity": h, "condition": c}))
        return self._send(404, json.dumps({"error": "unknown city"}))


def start_stub(port=8771):
    server = ThreadingHTTPServer(("127.0.0.1", port), Handler)
    threading.Thread(target=server.serve_forever, daemon=True).start()
    return f"http://127.0.0.1:{port}"
```

Real output

```
Chennai      ok after 1 attempt(s): 33.5 C, humidity 70%
Mumbai       ok after 1 attempt(s): 31.0 C, humidity 78%
Delhi        ok after 1 attempt(s): 24.5 C, humidity 40%
Bengaluru    ok after 2 attempt(s): 26.0 C, humidity 60%
Atlantis     FAILED, request rejected: HTTP 404
Brokenville  FAILED, invalid response: the answer is not JSON
Nulltown     FAILED, invalid response: missing field 'temp_c'
Hotville     FAILED, invalid response: temperature 999 C is impossible
Slowtown     FAILED, network problem: ReadTimeout

--- summary.txt, read back from the file ---
Cities requested : 9
Successful       : 4
Failed           : 5
Average temp     : 28.8 C
Hottest / coldest: Chennai 33.5 C / Delhi 24.5 C
Average humidity : 62.0 %
Problems:
  Atlantis: request rejected: HTTP 404
  Brokenville: invalid response: the answer is not JSON
  Nulltown: invalid response: missing field 'temp_c'
  Hotville: invalid response: temperature 999 C is impossible
  Slowtown: network problem: ReadTimeout
files written: ['readings.json', 'summary.txt']
```

M1-Q3 Intelligent Student Support Chatbot using NLTK and ChatterBot

3. Intelligent Student Support Chatbot

A university plans to develop a chatbot using **NLTK** and **ChatterBot** to answer student queries regarding courses, examinations, and placements. The chatbot should:

- Process user input and identify keywords.
- Store frequently asked questions and responses.
- Handle invalid or ambiguous queries.
- Continuously learn from user interactions.

Question: Propose an object-oriented design for the chatbot by identifying suitable classes, attributes, and methods. Explain how NLTK, ChatterBot, exception handling, and appropriate data structures can be integrated to improve chatbot accuracy and user experience.

The question as given (Sample Questions/6260159273853719039.jpg)

Answer

Object-oriented design: classes, attributes and methods

Class	Attributes	Methods	Responsibility
`Preprocessor`	`stop` (set of stop words), `stem` (stemmer)	`keywords(text)`	NLTK: tokenise, lower-case, drop stop words and punctuation, stem; raises `EmptyQueryError`
`KnowledgeBase`	`answers` (dict intent -> answer), `index` (dict keyword -> set of intents)	`add(intent, examples, answer)`, `rank(keywords)`	stores the FAQ and finds the intents that share keywords with a question
`SupportBot`	`kb`, `prep`, `brain` (ChatterBot), `history` (list), `unanswered` (Counter), `review_queue` (deque)	`ask(text)`, `learn(question, answer)`	the controller: chooses between FAQ, ChatterBot, clarification and fallback
`Interaction`	`query`, `answer`, `source`	(a dataclass)	one line of the conversation history
`EmptyQueryError`			custom exception for empty input

1. Process user input and identify keywords

NLTK's `word_tokenize` splits the text, stop words ("which", "and", "are", "there") and punctuation are removed, and `PorterStemmer` reduces words to their stem, so "exams" and "exam" match. "Which courses and exams are there?" becomes the set {`cours`, `exam`} (printed in the output). The same preprocessing is applied to the stored example questions, so both sides use the same vocabulary.

2. Store frequently asked questions and responses

- In the **KnowledgeBase**: a dictionary intent -> answer, plus an **inverted index** (keyword -> set of intents), so that finding the candidate answers for a question costs a few dictionary look-ups, not a scan of all FAQs.
- In **ChatterBot**: question-answer pairs trained with `ListTrainer` and stored in its SQLite database. It finds the stored question that is most similar to the user's wording, which covers phrasings the keyword index misses (the library timings in the output).

3. Handle invalid or ambiguous queries

Situation	Example in the run	Reaction
Empty input	`' '`	`EmptyQueryError` is raised in `keywords()` and caught in `ask()`: "Please type your question."
Ambiguous: two intents fit equally	"Which courses and exams are there?"	ask instead of guessing: "Do you mean courses or exams?"
Unknown	"How is the weather on campus?"	polite fallback; the question goes to a review queue and is counted

4. Continuously learn from user interactions

- Unknown questions are collected in a **deque** (`review_queue`, first in first out) and counted in a `Counter`, so staff can answer the most frequent ones first.

- A staff answer is added with `learn(question, answer)`, which calls `ListTrainer.train`. In the output the same question is answered correctly after that.
- Why not learn from everything automatically? ChatterBot can store every conversation, but then wrong or abusive inputs become "knowledge". Human-in-the-loop learning keeps accuracy under control (the demo bot is created with `read_only=True`).

How NLTK, ChatterBot, exceptions and data structures improve accuracy and user experience

Part	Contribution
NLTK preprocessing	stemming and stop-word removal make "exam", "exams" and "Exam dates?" meet; fewer false matches
Two-layer matching	exact keyword match first (fast, predictable), ChatterBot similarity second (flexible)
Exceptions	bad input never crashes the conversation; every problem has a friendly reply
dict / set / Counter / deque / list	O(1) look-ups, no duplicates, frequency statistics, a fair review queue, a full history for analysis
Clarification and fallback	the bot asks or admits it does not know, instead of giving a wrong answer with confidence

Watch out. Honest limits: keyword matching does not understand meaning or synonyms, and ChatterBot is retrieval-based (it returns stored answers, it does not generate them). For better accuracy later, add an intent classifier (TF-IDF with logistic regression) or sentence embeddings.

Code, run in an environment with NLTK and ChatterBot

Code: code/sa_m1_q3.py

```
"""Sample answer M1-Q3: an object-oriented student-support chatbot built with NLTK, ChatterBot, exceptions and
    suitable data structures."""
import logging
import os
import tempfile
from collections import Counter, deque
from dataclasses import dataclass

from chatterbot import ChatBot
from chatterbot.trainers import ListTrainer
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

logging.disable(logging.CRITICAL)
os.chdir(tempfile.mkdtemp(prefix="sa_chatbot_"))


class EmptyQueryError(ValueError):
    """The user sent nothing usable."""


@dataclass
class Interaction:                                  # one line of the conversation history
    query: str
    answer: str
    source: str                                     # keywords / chatterbot / clarify / fallback / empty


class Preprocessor:                                 # NLTK: tokenise, remove stop words, stem
    def __init__(self):
        self.stop = set(stopwords.words("english"))
        self.stem = PorterStemmer().stem

    def keywords(self, text):
        if not text or not text.strip():
            raise EmptyQueryError("empty question")
        return {self.stem(t) for t in word_tokenize(text.lower()) if t.isalpha() and t not in self.stop}


class KnowledgeBase:                                # frequently asked questions: dict + inverted index of sets
    def __init__(self, prep):
        self.prep = prep
        self.answers = {}                           # intent -> answer
        self.index = {}                             # keyword -> set of intents

    def add(self, intent, example_questions, answer):
        self.answers[intent] = answer
        for q in example_questions:
            for kw in self.prep.keywords(q):
                self.index.setdefault(kw, set()).add(intent)

    def rank(self, keywords):                       # how many query keywords point to each intent
        score = Counter()
        for kw in keywords:
            for intent in self.index.get(kw, ()):
                score[intent] += 1
        return score.most_common()


class SupportBot:
    UNKNOWN = "__unknown__"

    def __init__(self):
        self.prep = Preprocessor()
        self.kb = KnowledgeBase(self.prep)
        self.history = []                           # list of Interaction objects
        self.unanswered = Counter()                 # how often each unknown question was asked
        self.review_queue = deque()                 # unknown questions waiting for a human answer (FIFO)
        self.brain = ChatBot("SupportBot", storage_adapter="chatterbot.storage.SQLStorageAdapter",
            database_uri="sqlite:///bot.sqlite3",
                             logic_adapters=[{"import_path": "chatterbot.logic.BestMatch", "default_response":
                                 self.UNKNOWN,
                                              "maximum_similarity_threshold": 0.90}], read_only=True)
        self.trainer = ListTrainer(self.brain)

    def ask(self, text):
        try:
            words = self.prep.keywords(text)
        except EmptyQueryError:
            return self._log(text, "Please type your question.", "empty")
        ranked = self.kb.rank(words)
        if ranked and (len(ranked) == 1 or ranked[0][1] > ranked[1][1]):
            return self._log(text, self.kb.answers[ranked[0][0]], "keywords")
        if len(ranked) > 1:                         # two intents fit equally well: ask instead of guessing
```

```
            options = " or ".join(i for i, score in ranked if score == ranked[0][1])
            return self._log(text, f"Do you mean {options}?", "clarify")
        reply = self.brain.get_response(text).text   # nothing in the FAQ: try ChatterBot's stored conversations
        if reply != self.UNKNOWN:
            return self._log(text, reply, "chatterbot")
        self.unanswered[text] += 1
        if self.unanswered[text] == 1:
            self.review_queue.append(text)
        return self._log(text, "I do not know yet. A staff member will answer this question.", "fallback")

    def learn(self, question, answer):              # supervised learning: only reviewed answers are trained
        self.trainer.train([question, answer])

    def _log(self, query, answer, source):
        self.history.append(Interaction(query, answer, source))
        return answer


bot = SupportBot()
bot.kb.add("exams", ["When is the semester exam timetable?", "Exam dates and hall tickets"], "Exams start on 4
    December. Hall tickets are on the student portal.")
bot.kb.add("courses", ["Which courses are offered?", "How do I register for a course?"], "Course registration is
    open on the student portal until 20 November.")
bot.kb.add("placements", ["Which companies visit for placement?", "How do I register for the placement drive?"],
    "Register for placements on the career portal.")
bot.trainer.train(["What are the library timings?", "The library is open from 9 am to 8 pm."])

for q in ["When are the exams?", "Tell me about placement companies", "Which courses and exams are there?", "   ",
          "What are the library timings?", "How is the weather on campus?"]:
    print(f"{q!r:<42} -> {bot.ask(q)}")

print("\n--- continuous learning with a human in the loop ---")
print("waiting for review:", list(bot.review_queue))
question = bot.review_queue.popleft()
bot.learn(question, "Please check the campus notice board for weather alerts.")
print(f"{question!r} -> {bot.ask(question)}")

print("\n--- statistics from the history ---")
print("answers by source:", dict(Counter(i.source for i in bot.history)))
print("keywords of 'Which courses and exams are there?':", sorted(bot.prep.keywords("Which courses and exams are
    there?")))
print("questions still unanswered:", dict(bot.unanswered), "| queue length:", len(bot.review_queue))
```

Real output

```
'When are the exams?'                      -> Exams start on 4 December. Hall tickets are on the student portal.
'Tell me about placement companies'        -> Register for placements on the career portal.
'Which courses and exams are there?'       -> Do you mean courses or exams?
'   '                                      -> Please type your question.
'What are the library timings?'            -> The library is open from 9 am to 8 pm.
'How is the weather on campus?'            -> I do not know yet. A staff member will answer this question.

--- continuous learning with a human in the loop ---
waiting for review: ['How is the weather on campus?']
'How is the weather on campus?' -> Please check the campus notice board for weather alerts.

--- statistics from the history ---
answers by source: {'keywords': 2, 'clarify': 1, 'empty': 1, 'chatterbot': 2, 'fallback': 1}
keywords of 'Which courses and exams are there?': ['cours', 'exam']
questions still unanswered: {'How is the weather on campus?': 1} | queue length: 0
```

Module 2: Python libraries for data science

5 sample questions with model answers.

M2-Q1 Student Performance Analysis using Pandas

Question 1: Student Performance Analysis using Pandas

A university has collected student performance data as shown below.

Dataset

StudentID	Name	Department	Attendance (%)	Internal Marks	Final Marks
S101	Arun	CSE	92	45	78
S102	Priya	ECE	85	38	72
S103	Kiran	CSE	NaN	42	80
S104	Meena	IT	78	NaN	68
S105	Ravi	CSE	95	48	88

HOT Question

As a Data Analyst, design a data preprocessing strategy to prepare the dataset for predictive modeling. Explain:

- How missing values should be handled.
- Which Pandas built-in functions can be used for statistical analysis.
- How you would identify students at academic risk.
- Which visualizations would best communicate the findings to faculty members.

The question as given (Sample Questions/6267292660481594230.jpg)

Answer, point by point

1. How should missing values be handled?

- **Look first, fix second.** `df.isna().sum()` shows one missing Attendance (S103) and one missing Internal Marks (S104). Two of five rows (40%) have a gap, so dropping rows would throw away almost half of the data. Do not use `dropna()` here.
- **Impute numeric columns with the median**, not the mean: the median is not pulled by one extreme student. Use the median of the **same department** when it exists, otherwise the overall median. S103 (CSE) gets 93.5, the median of the other CSE students (92 and 95). S104 is the only IT student, so there is no department median and the overall median 43.5 is used.
- **Keep a flag** (`Attendance_filled`, `InternalMarks_filled`) so that the faculty and the model can see which values were estimated.
- **Never impute the target** (Final Marks). For modelling, put `SimpleImputer(strategy='median')` inside a scikit-learn `Pipeline` and fit it on the training rows only, otherwise test-set information leaks into the model (the code shows the training-only version).
- Best of all, go back to the source: a missing attendance figure is usually a data-entry gap that the registrar can fill.

2. Which Pandas built-in functions can be used for statistical analysis?

Need	Function	Result on this data
Summary of every numeric column	`df.describe()`	mean attendance 88.7, mean internal 43.3, mean final 77.2
Relationship between marks	`df[cols].corr()`	attendance~final 0.92, internal~final 0.65
Group comparison	`df.groupby('Department')['FinalMarks'].agg(['count','mean','max'])`	CSE 82.0 (3 students), ECE 72.0, IT 68.0
Gaps and quality	`isna().sum()`, `duplicated()`, `dtypes`	2 gaps, 0 duplicates
Ranking and bands	`sort_values`, `nlargest`, `rank`, `pd.cut`, `quantile`	used for the risk band below

3. How would you identify students at academic risk?

Write the rule down so that it can be explained and changed. Example rule (thresholds are for the faculty to set): one point for attendance below 80%, one for internal marks below 40, one for final marks below 70. 0 points = ok, 1 = watch, 2 or more = at risk.

- **Meena** (78% attendance, final 68, filled internal mark) gets 2 points: **at risk**.
- **Priya** (internal 38) gets 1 point: **watch**. The other three have 0 points.
- With more data replace the hand-made rule by a model (logistic regression or a decision tree on the history of earlier batches) and check it with cross-validation. With five students any model would be guesswork.

4. Which visualizations communicate the findings best to faculty?

- **Scatter plot** of attendance against final marks (r = 0.92), one colour per department, at-risk students labelled: shows the relationship and who is off the pattern.
- **Bar chart** of mean final marks per department: one number per group, easy to read.
- **Box plots** of marks per department once there are enough students; a **correlation heat map** for the analyst; a plain **table with the risk status highlighted** for a meeting.
- Avoid pie charts and 3-D charts; give every chart a title, axis labels and one message.

Code, run on the dataset of the question

Code: code/sa_m2_q1.py

```
"""Sample answer M2-Q1: preprocessing, statistics and risk detection for the student performance table."""
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer

df = pd.DataFrame({
    "StudentID": ["S101", "S102", "S103", "S104", "S105"],
    "Name": ["Arun", "Priya", "Kiran", "Meena", "Ravi"],
    "Department": ["CSE", "ECE", "CSE", "IT", "CSE"],
    "Attendance": [92, 85, np.nan, 78, 95],
    "InternalMarks": [45, 38, 42, np.nan, 48],
    "FinalMarks": [78, 72, 80, 68, 88]})
raw = df.copy()

print("--- 1. inspect the missing values first ---")
print(df.isna().sum().to_dict(), "| share of rows with a gap:", round(df.isna().any(axis=1).mean(), 2))

# keep a flag so that nobody forgets which numbers were filled in
for col in ["Attendance", "InternalMarks"]:
    df[col + "_filled"] = df[col].isna()
    dept_median = df.groupby("Department")[col].transform("median")      # median of the same department
    df[col] = df[col].fillna(dept_median).fillna(df[col].median())        # fall back to the overall median
print(df[["StudentID", "Department", "Attendance", "InternalMarks"]])

print("\n--- the same idea inside a modelling pipeline (fit on training rows only) ---")
imp = SimpleImputer(strategy="median").fit(raw.loc[:3, ["Attendance", "InternalMarks"]])     # rows 0-3 play the
    training set
print(imp.transform(raw[["Attendance", "InternalMarks"]]))

print("\n--- 2. built-in functions for statistical analysis ---")
num = ["Attendance", "InternalMarks", "FinalMarks"]
print(df[num].describe().round(2))
print(df[num].corr().round(2))
print(df.groupby("Department")["FinalMarks"].agg(["count", "mean", "max"]))

print("\n--- 3. who is at academic risk? (example thresholds, set them with the faculty) ---")
df["risk_points"] = (df["Attendance"] < 80).astype(int) + (df["InternalMarks"] < 40).astype(int) + (df["FinalMarks"]
     < 70).astype(int)
df["status"] = pd.cut(df["risk_points"], bins=[-1, 0, 1, 3], labels=["ok", "watch", "at risk"])
print(df[["Name", "Attendance", "InternalMarks", "FinalMarks", "risk_points", "status"]])

fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
for dept, g in df.groupby("Department"):
    ax[0].scatter(g["Attendance"], g["FinalMarks"], s=60, label=dept)
for _, r in df[df["risk_points"] > 0].iterrows():
    ax[0].annotate(r["Name"], (r["Attendance"], r["FinalMarks"]), textcoords="offset points", xytext=(6, 4))
ax[0].set_xlabel("Attendance (%)")
ax[0].set_ylabel("Final marks")
ax[0].set_title("Attendance vs final marks")
ax[0].legend()
df.groupby("Department")["FinalMarks"].mean().plot.bar(ax=ax[1], color="steelblue", rot=0)
ax[1].set_ylabel("Mean final marks")
ax[1].set_title("Mean final marks by department")
plt.tight_layout()
plt.savefig("sa_m2_q1.png", dpi=110)
print("\nsaved sa_m2_q1.png")
```

Real output

```
--- 1. inspect the missing values first ---
{'StudentID': 0, 'Name': 0, 'Department': 0, 'Attendance': 1, 'InternalMarks': 1, 'FinalMarks': 0} | share of rows
    with a gap: 0.4
  StudentID Department  Attendance  InternalMarks
0      S101        CSE        92.0           45.0
1      S102        ECE        85.0           38.0
2      S103        CSE        93.5           42.0
3      S104         IT        78.0           43.5
4      S105        CSE        95.0           48.0

--- the same idea inside a modelling pipeline (fit on training rows only) ---
[[92. 45.]
 [85. 38.]
 [85. 42.]
 [78. 42.]
 [95. 48.]]

--- 2. built-in functions for statistical analysis ---
       Attendance  InternalMarks  FinalMarks
count         5.0            5.0        5.00
mean         88.7           43.3       77.20
std           7.1            3.7        7.69
min          78.0           38.0       68.00
25%          85.0           42.0       72.00
50%          92.0           43.5       78.00
75%          93.5           45.0       80.00
max          95.0           48.0       88.00
               Attendance  InternalMarks  FinalMarks
Attendance           1.00           0.44        0.92
InternalMarks        0.44           1.00        0.65
FinalMarks           0.92           0.65        1.00
            count  mean  max
Department
CSE             3  82.0   88
ECE             1  72.0   72
IT              1  68.0   68

--- 3. who is at academic risk? (example thresholds, set them with the faculty) ---
    Name  Attendance  InternalMarks  FinalMarks  risk_points   status
0   Arun        92.0           45.0          78            0       ok
1  Priya        85.0           38.0          72            1    watch
2  Kiran        93.5           42.0          80            0       ok
3  Meena        78.0           43.5          68            2  at risk
4   Ravi        95.0           48.0          88            0       ok

saved sa_m2_q1.png
```

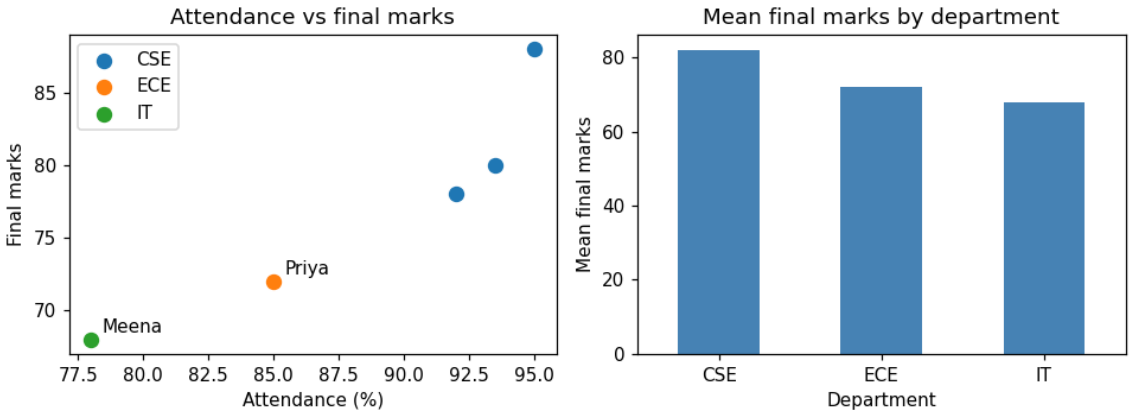
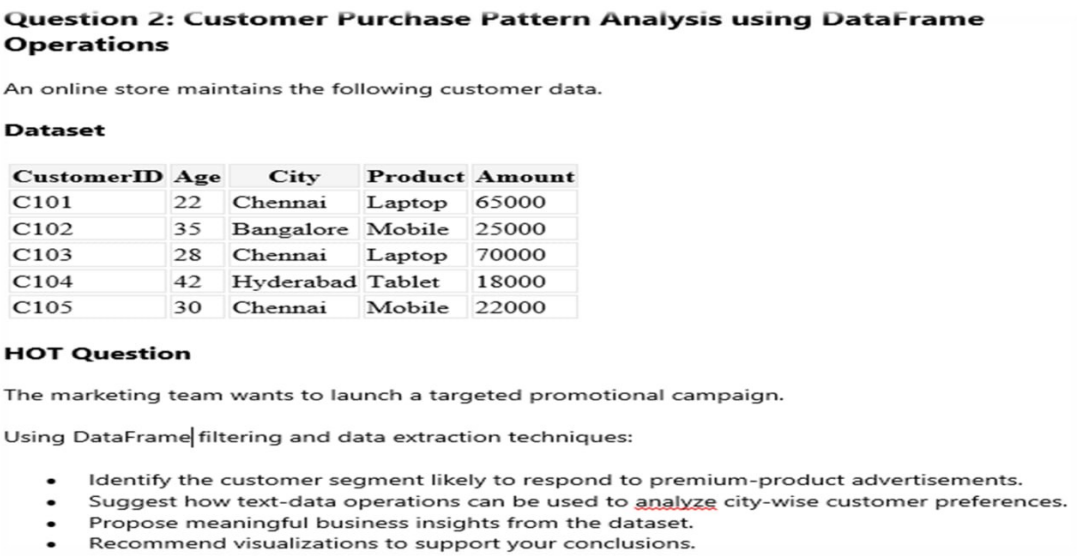


Figure produced by the code: attendance against final marks (left), mean final marks per department (right).

M2-Q2 Customer Purchase Pattern Analysis using DataFrame Operations



Question 2: Customer Purchase Pattern Analysis using DataFrame Operations

An online store maintains the following customer data.

Dataset

CustomerID	Age	City	Product	Amount
C101	22	Chennai	Laptop	65000
C102	35	Bangalore	Mobile	25000
C103	28	Chennai	Laptop	70000
C104	42	Hyderabad	Tablet	18000
C105	30	Chennai	Mobile	22000

HOT Question

The marketing team wants to launch a targeted promotional campaign.

Using DataFrame filtering and data extraction techniques:

- Identify the customer segment likely to respond to premium-product advertisements.
- Suggest how text-data operations can be used to analyze city-wise customer preferences.
- Propose meaningful business insights from the dataset.
- Recommend visualizations to support your conclusions.

The question as given (Sample Questions/6267292660481594244.jpg)

Answer, point by point

1. Which customer segment is likely to respond to premium-product advertisements?

Define "premium" with the data instead of guessing: the top quarter of the order values, `threshold = df['Amount'].quantile(0.75)` = Rs. 65,000. Boolean filtering (`df[df['Amount'] >= threshold]`, or `query`, or `loc`) returns two customers:

CustomerID	Age	City	Product	Amount
C101	22	Chennai	Laptop	65,000
C103	28	Chennai	Laptop	70,000

Segment: **young (mean age 25) Chennai customers who buy laptops**. Extension for the campaign: all Chennai customers, plus anyone who bought a laptop or tablet. Two customers are an anecdote, not a proof: on real data build the segment from recency, frequency and spend and test the advertisement on a small group first.

2. How can text-data operations analyse city-wise preferences?

- **Normalise** the text before grouping: `str.strip()`, `str.title()`, `replace({'Bengaluru': 'Bangalore'})`. Otherwise " chennai" and "CHENNAI " become two cities.
- **Search** with `str.contains('^Chen', case=False, regex=True)`, **pull out** parts with `str.extract(r'(Chennai|Bangalore)')`, split with `str.split`, measure with `str.len`.
- **Count**: `pd.crosstab(df['City'], df['Product'])` gives Chennai 2 laptops and 1 mobile, Bangalore 1 mobile, Hyderabad 1 tablet. The most bought product per city is then `groupby('City')['Product'].agg(lambda s: s.mode().iat[0])`.
- The same functions work on free-text reviews: find the product or city mentioned in each review and count by city.

3. What business insights can be proposed?

- **Chennai matters most**: 3 of 5 customers (60%) and Rs. 157,000 of the Rs. 200,000 revenue (78.5%).
- **Laptops carry the revenue**: 67.5% of revenue from 2 of 5 orders; mobiles 23.5%, tablets 9.0%. The average order is Rs. 40,000.
- **Younger customers spend more**: correlation of age and amount is -0.79, but with five rows this is only a hint.
- **Actions**: premium laptop campaign for young Chennai buyers, accessories bundled with laptops, mobile offers to move Bangalore and Chennai buyers up the range, a test campaign in Hyderabad where only one tablet was sold.

4. Which visualizations support the conclusions?

- **Bar chart** of revenue by product (comparison of categories).
- **Stacked bar chart** of orders by city and product (composition).

- **Scatter plot** of age against amount coloured by product (relationship).
- A **map** (Folium) for city-level revenue once coordinates are available.

Code, run on the dataset of the question

Code: code/sa_m2_q2.py

```
"""Sample answer M2-Q2: filtering, text operations and insights for the customer purchase table."""
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "CustomerID": ["C101", "C102", "C103", "C104", "C105"],
    "Age": [22, 35, 28, 42, 30],
    "City": ["Chennai", "Bangalore", "Chennai", "Hyderabad", "Chennai"],
    "Product": ["Laptop", "Mobile", "Laptop", "Tablet", "Mobile"],
    "Amount": [65000, 25000, 70000, 18000, 22000]})

print("--- 1. the premium-product segment ---")
threshold = df["Amount"].quantile(0.75)                      # top quarter of the order values
premium = df[df["Amount"] >= threshold]
print("threshold:", threshold)
print(premium[["CustomerID", "Age", "City", "Product", "Amount"]])
print("same result with query():", df.query("Amount >= @threshold")["CustomerID"].tolist())
print("segment profile: mean age", premium["Age"].mean(), "| cities", premium["City"].unique().tolist(), "|
    products", premium["Product"].unique().tolist())
print("laptop buyers by extraction:", df.loc[df["Product"] == "Laptop", ["CustomerID", "City"]].values.tolist())

print("\n--- 2. text operations ---")
messy = pd.Series([" chennai", "CHENNAI ", "Bangalore", "bengaluru", "hyderabad"])
tidy = messy.str.strip().str.title().replace({"Bengaluru": "Bangalore"})
print(tidy.tolist())
print("starts with Chen:", df["City"].str.contains("^Chen", case=False, regex=True).tolist())
reviews = pd.Series(["Great laptop, fast delivery in Chennai", "Mobile screen cracked, Bangalore store", "Tablet is
    fine"])
print("mentions laptop:", reviews.str.contains("laptop", case=False).tolist())
print("city found in text:", reviews.str.extract(r"(Chennai|Bangalore|Hyderabad)")[0].tolist())
print(pd.crosstab(df["City"], df["Product"]))
print("most bought product per city:")
print(df.groupby("City")["Product"].agg(lambda s: s.mode().iat[0]))

print("\n--- 3. business insights ---")
by_city = df.groupby("City")["Amount"].agg(orders="count", revenue="sum", average="mean")
by_product = df.groupby("Product")["Amount"].sum().sort_values(ascending=False)
print(by_city)
print((by_product / by_product.sum() * 100).round(1).to_dict(), "(% of revenue)")
print("average order value:", df["Amount"].mean(), "| Chennai share of customers:", round((df["City"] ==
    "Chennai").mean() * 100), "%")
print("correlation age vs amount:", round(df["Age"].corr(df["Amount"]), 2), "(only 5 rows, treat as a hint)")

fig, ax = plt.subplots(1, 3, figsize=(11, 3.2))
by_product.plot.bar(ax=ax[0], color="steelblue", rot=0)
ax[0].set_title("Revenue by product")
pd.crosstab(df["City"], df["Product"]).plot.bar(stacked=True, ax=ax[1], rot=0)
ax[1].set_title("Orders by city and product")
for product, g in df.groupby("Product"):
    ax[2].scatter(g["Age"], g["Amount"], s=60, label=product)
ax[2].set_xlabel("Age")
ax[2].set_ylabel("Amount")
ax[2].set_title("Age vs amount")
ax[2].legend()
plt.tight_layout()
plt.savefig("sa_m2_q2.png", dpi=110)
print("\nsaved sa_m2_q2.png")
```

Real output

```
--- 1. the premium-product segment ---
threshold: 65000.0
  CustomerID  Age     City Product  Amount
0       C101   22  Chennai  Laptop   65000
2       C103   28  Chennai  Laptop   70000
same result with query(): ['C101', 'C103']
segment profile: mean age 25.0 | cities ['Chennai'] | products ['Laptop']
laptop buyers by extraction: [['C101', 'Chennai'], ['C103', 'Chennai']]

--- 2. text operations ---
['Chennai', 'Chennai', 'Bangalore', 'Bangalore', 'Hyderabad']
starts with Chen: [True, False, True, False, True]
mentions laptop: [True, False, False]
city found in text: ['Chennai', 'Bangalore', nan]
Product    Laptop  Mobile  Tablet
City
Bangalore       0       1       0
Chennai         2       1       0
Hyderabad       0       0       1
most bought product per city:
City
Bangalore    Mobile
Chennai      Laptop
Hyderabad    Tablet
Name: Product, dtype: object

--- 3. business insights ---
           orders  revenue       average
City
Bangalore       1    25000  25000.000000
Chennai         3   157000  52333.333333
Hyderabad       1    18000  18000.000000
{'Laptop': 67.5, 'Mobile': 23.5, 'Tablet': 9.0} (% of revenue)
average order value: 40000.0 | Chennai share of customers: 60 %
correlation age vs amount: -0.79 (only 5 rows, treat as a hint)

saved sa_m2_q2.png
```

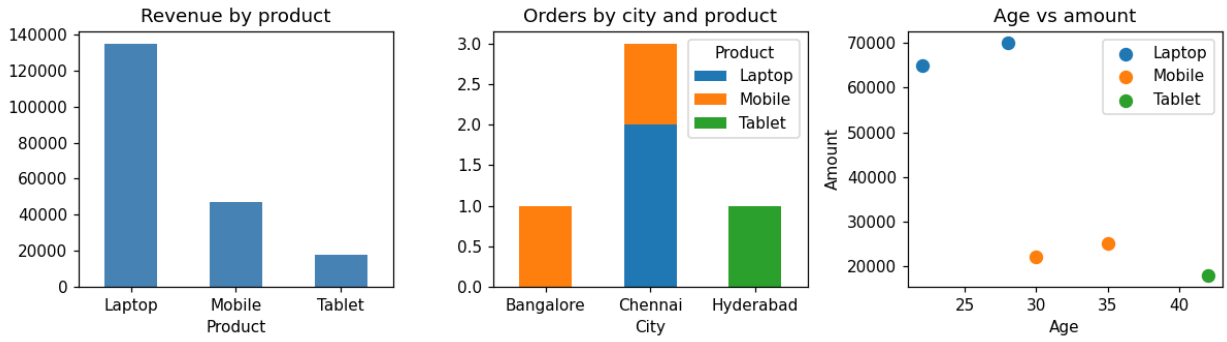
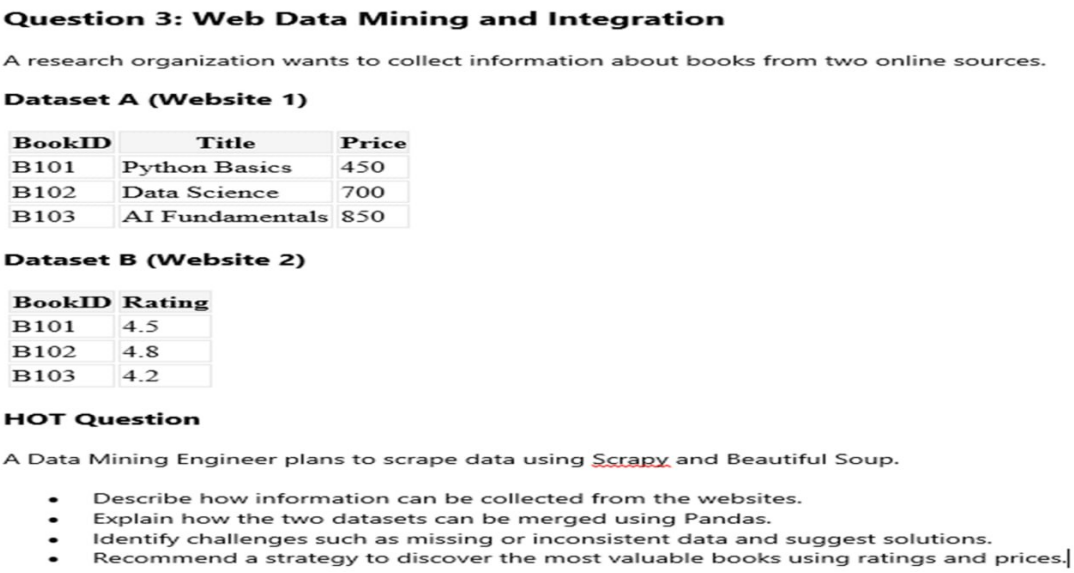


Figure produced by the code: revenue by product, orders by city and product, age against amount.

M2-Q3 Web Data Mining and Integration (Scrapy, Beautiful Soup, merging)



Question 3: Web Data Mining and Integration

A research organization wants to collect information about books from two online sources.

Dataset A (Website 1)

BookID	Title	Price
B101	Python Basics	450
B102	Data Science	700
B103	AI Fundamentals	850

Dataset B (Website 2)

BookID	Rating
B101	4.5
B102	4.8
B103	4.2

HOT Question

A Data Mining Engineer plans to scrape data using Scrapy and Beautiful Soup.

- Describe how information can be collected from the websites.
- Explain how the two datasets can be merged using Pandas.
- Identify challenges such as missing or inconsistent data and suggest solutions.
- Recommend a strategy to discover the most valuable books using ratings and prices.

The question as given (Sample Questions/6267292660481594245.jpg)

Answer, point by point

1. How can information be collected from the websites?

- **Check first**: the site's terms of use and `robots.txt`; look for an official API or a downloadable file, which is more reliable than scraping.
- **Inspect the page** (browser developer tools) to find the repeating element, e.g. one `div.book` per book with the id, the title and the price inside it.
- **Fetch** the HTML with `requests.get(url, timeout=10)` and **parse** it with Beautiful Soup (`find_all`, `select`) or let a **Scrapy** spider do both with CSS or XPath selectors.
- **Be polite**: a delay between requests, a clear user-agent, obey robots.txt (Scrapy: `DOWNLOAD_DELAY`, `AUTOTHROTTLE_ENABLED`, `ROBOTSTXT_OBEY`).
- **Store** the result as CSV/JSON/DataFrame (Scrapy: the `FEEDS` setting).

	Beautiful Soup (with requests)	Scrapy
What it is	HTML parsing library; you write the loop	crawling framework: scheduler, downloader, pipelines, feeds
Best for	one page or a few pages, quick experiments	many pages, pagination, several sites, regular runs
Politeness, retries, export	write it yourself	built in (settings)
Here	one function per website (code below)	one spider per website (second code block)

2. How can the two datasets be merged using Pandas?

Both tables share the key `BookID`, so `prices.merge(ratings, on='BookID', how='left', validate='one_to_one', indicator=True)`. Choose the join on purpose: **inner** keeps books found on both sites, **left** keeps every priced book and shows missing ratings as NaN, **outer** keeps everything. `validate='one_to_one'` stops silently duplicated rows, `indicator=True` adds a `_merge` column that audits which side each row came from.

> **Watch out.** The keys must be **identical** before the merge. In the code the second site writes the first id as ' b101 ' (lower case, spaces), so the raw key sets are not equal. `str.strip().str.upper()` fixes it. A merge on the uncleaned keys would quietly lose that book.

3. What challenges (missing or inconsistent data) occur and what are the solutions?

Challenge	Example	Solution
Text instead of numbers	'Rs. 450', 'N/A'	extract the number with a regex, `pd.to_numeric(errors='coerce')`
Mismatching keys	' b101 ' against 'B101'	`str.strip().str.upper()` on both sides, then merge
Duplicates	B102 listed twice	`drop_duplicates(subset='BookID')`
Missing values	no rating for B102, price 'N/A' for B104	inspect with `isna()`, then drop, impute or flag; do not rank books without a rating
Unmatched ids	B104 only on site 1, B105 only on site 2	outer merge with `indicator=True`, review the `left_only` / `right_only` rows
Different scales or currencies	ratings out of 5 and out of 10	rescale to one scale before comparing
Site changes or blocking	HTML layout changes, HTTP 429	tests on the selectors, retries with back-off, logging of failed URLs

Tip. A trap found while testing this answer: the cleaning pattern `[^0-9.]` keeps the full stop of "Rs.", so "Rs. 450" became ".450" and then 0.45. Extract the first number instead: `str.extract(r'(\d+(?:\.\d+)?)')`.

4. Which strategy discovers the most valuable books using ratings and prices?

"Valuable" must be defined before it is computed. Three definitions give different winners on this data:

Definition	Formula	Winner
Best value for money	rating / price x 100: B101 1.000, B102 0.686, B103 0.494	**Python Basics**
Best trade-off (weights 60% rating, 40% price, both min-max scaled)	B101 0.70, B102 0.75, B103 0.00	**Data Science**
Highest quality	highest rating 4.8	**Data Science**

- **Pareto view**: B101 is cheaper, B102 is better rated, and neither beats the other. B103 costs more and is rated lower than both, so it is dominated and can be dropped.
- Ratings from few reviews are unreliable: add the number of reviews and use a Bayesian (shrunk) average before ranking.
- Recommendation: show the Pareto set (B101 and B102) and let the buyer's budget decide.

Code 1: Beautiful Soup, cleaning, merge and ranking

Code: code/sa_m2_q3.py

```
"""Sample answer M2-Q3: collect two tables with Beautiful Soup, clean and merge them with pandas, rank the books by
    value."""
import pandas as pd
from bs4 import BeautifulSoup

# Stand-ins for the HTML that requests.get(url).text would return from the two websites.
SITE1 = """<html><body>
<div class="book" data-id="B101"><h3>Python Basics</h3><span class="price">Rs. 450</span></div>
<div class="book" data-id="B102"><h3>Data Science</h3><span class="price">Rs. 700</span></div>
<div class="book" data-id="B103"><h3>AI Fundamentals</h3><span class="price">Rs. 850</span></div>
</body></html>"""
SITE2 = """<ul>
<li data-id=" b101 "><span class="rating">4.5</span></li>
<li data-id="B102"><span class="rating">4.8</span></li>
<li data-id="B103"><span class="rating">4.2</span></li>
</ul>"""


def scrape_prices(html):
    soup = BeautifulSoup(html, "html.parser")
    return pd.DataFrame([{"BookID": d["data-id"], "Title": d.h3.text, "Price": d.find("span", class_="price").text}
                         for d in soup.find_all("div", class_="book")])


def scrape_ratings(html):
    soup = BeautifulSoup(html, "html.parser")
    return pd.DataFrame([{"BookID": li["data-id"], "Rating": li.find("span", class_="rating").text} for li in
        soup.find_all("li")])


def clean(df):
    df = df.copy()
    df["BookID"] = df["BookID"].str.strip().str.upper()                    # keys must match exactly before a merge
    for col in ("Price", "Rating"):
        if col in df:                                  # take the first number in the text; "N/A" and None become
            NaN
            df[col] = pd.to_numeric(df[col].astype(str).str.extract(r"(\d+(?:\.\d+)?)")[0], errors="coerce")
    return df.drop_duplicates(subset="BookID")


print("--- scraped (raw) ---")
a, b = scrape_prices(SITE1), scrape_ratings(SITE2)
print(a, "\n", b, sep="")
print("keys equal before cleaning:", set(a["BookID"]) == set(b["BookID"]))

print("\n--- cleaned and merged ---")
a, b = clean(a), clean(b)
books = a.merge(b, on="BookID", how="left", validate="one_to_one", indicator=True)
print(books)
print(books["_merge"].value_counts().to_dict())

print("\n--- when the data is messy (missing price, duplicate, missing rating, unmatched id) ---")
dirty_a = pd.DataFrame({"BookID": ["B101", "B102", "B102", "B104"], "Price": ["Rs. 450", "700", "700", "N/A"]})
dirty_b = pd.DataFrame({"BookID": ["b101 ", "B102", "B105"], "Rating": ["4.5", None, "3.9"]})
m = clean(dirty_a).merge(clean(dirty_b), on="BookID", how="outer", indicator=True)
print(m)
print("rows with a missing value:", m[["Price", "Rating"]].isna().any(axis=1).sum(), "| only in one source:",
    (m["_merge"] != "both").sum())
usable = m.dropna(subset=["Price", "Rating"])
print("usable rows:", usable["BookID"].tolist())

print("\n--- which books are the most valuable? it depends on the definition ---")
minmax = lambda s: (s - s.min()) / (s.max() - s.min())
books["rating_per_100_rs"] = (books["Rating"] / books["Price"] * 100).round(3)
books["weighted_score"] = (0.6 * minmax(books["Rating"]) + 0.4 * (1 - minmax(books["Price"]))).round(3)
print(books[["BookID", "Title", "Price", "Rating", "rating_per_100_rs", "weighted_score"]])
print("best value for money :", books.loc[books["rating_per_100_rs"].idxmax(), "Title"])
print("best weighted score  :", books.loc[books["weighted_score"].idxmax(), "Title"])
print("highest rated        :", books.loc[books["Rating"].idxmax(), "Title"])
```

Real output

```
--- scraped (raw) ---
  BookID           Title    Price
0   B101    Python Basics  Rs. 450
1   B102     Data Science  Rs. 700
2   B103  AI Fundamentals  Rs. 850
   BookID Rating
0   b101     4.5
1    B102    4.8
2    B103    4.2
keys equal before cleaning: False

--- cleaned and merged ---
  BookID           Title  Price  Rating _merge
0   B101    Python Basics    450     4.5   both
1   B102     Data Science    700     4.8   both
2   B103  AI Fundamentals    850     4.2   both
{'both': 3, 'left_only': 0, 'right_only': 0}

--- when the data is messy (missing price, duplicate, missing rating, unmatched id) ---
  BookID  Price  Rating      _merge
0   B101  450.0     4.5        both
1   B102  700.0     NaN        both
2   B104    NaN     NaN   left_only
3   B105    NaN     3.9  right_only
rows with a missing value: 3 | only in one source: 2
usable rows: ['B101']

--- which books are the most valuable? it depends on the definition ---
  BookID           Title  Price  Rating  rating_per_100_rs  weighted_score
0   B101    Python Basics    450     4.5              1.000            0.70
1   B102     Data Science    700     4.8              0.686            0.75
2   B103  AI Fundamentals    850     4.2              0.494            0.00
best value for money : Python Basics
best weighted score  : Data Science
highest rated        : Data Science
```

Code 2: the same collection with Scrapy (two spiders)

Code: code/sa_m2_q3_scrapy.py (runs on local copies of the two pages)

```
"""Sample answer M2-Q3 (collection with Scrapy): one spider per website.
Local copies of the two pages are used (file:// URLs), so no website is contacted."""
import json
import os
import tempfile
from pathlib import Path

import pandas as pd
import scrapy
from scrapy.crawler import CrawlerProcess

os.chdir(tempfile.mkdtemp(prefix="sa_scrapy_"))
Path("site1.html").write_text("""<html><body>
<div class="book" data-id="B101"><h3>Python Basics</h3><span class="price">Rs. 450</span></div>
<div class="book" data-id="B102"><h3>Data Science</h3><span class="price">Rs. 700</span></div>
<div class="book" data-id="B103"><h3>AI Fundamentals</h3><span class="price">Rs. 850</span></div>
</body></html>""", encoding="utf-8")
Path("site2.html").write_text("""<ul>
<li data-id="B101"><span class="rating">4.5</span></li>
<li data-id="B102"><span class="rating">4.8</span></li>
<li data-id="B103"><span class="rating">4.2</span></li>
</ul>""", encoding="utf-8")

POLITE = {"LOG_LEVEL": "ERROR", "ROBOTSTXT_OBEY": True,        # respect robots.txt
          "DOWNLOAD_DELAY": 1, "AUTOTHROTTLE_ENABLED": True}   # be polite to the server


class PriceSpider(scrapy.Spider):                              # website 1: id, title, price
    name = "prices"
    start_urls = [Path("site1.html").resolve().as_uri()]       # in real life: the URL of website 1
    custom_settings = {**POLITE, "FEEDS": {"prices.json": {"format": "json", "overwrite": True}}}

    def parse(self, response):
        for d in response.css("div.book"):
            yield {"BookID": d.attrib["data-id"], "Title": d.css("h3::text").get(),
                   "Price": float(d.css("span.price::text").get().replace("Rs.", "").strip())}


class RatingSpider(scrapy.Spider):                             # website 2: id, rating
    name = "ratings"
    start_urls = [Path("site2.html").resolve().as_uri()]       # in real life: the URL of website 2
    custom_settings = {**POLITE, "FEEDS": {"ratings.json": {"format": "json", "overwrite": True}}}

    def parse(self, response):
        for li in response.css("li"):
            yield {"BookID": li.attrib["data-id"], "Rating": float(li.css("span.rating::text").get())}


process = CrawlerProcess()
process.crawl(PriceSpider)
process.crawl(RatingSpider)
process.start()                                                # runs both spiders, returns when both are done

prices = pd.DataFrame(json.loads(Path("prices.json").read_text(encoding="utf-8")))
ratings = pd.DataFrame(json.loads(Path("ratings.json").read_text(encoding="utf-8")))
print("items collected:", len(prices), "prices and", len(ratings), "ratings")
print(prices.merge(ratings, on="BookID", validate="one_to_one"))
```

Real output

```
items collected: 3 prices and 3 ratings
  BookID            Title  Price  Rating
0   B101    Python Basics  450.0     4.5
1   B102     Data Science  700.0     4.8
2   B103  AI Fundamentals  850.0     4.2
```

M2-Q4 Predicting House Prices using Machine Learning

Question 4: Predicting House Prices using Machine Learning

A real-estate company has collected the following housing dataset.

Dataset

HouseSize (sq.ft.)	Bedrooms	Age (Years)	Price (Lakhs)
1200	2	10	45
1500	3	8	60
1800	3	5	75
2200	4	3	95
2500	4	2	110

HOT Question

The company wants to develop a predictive model using NumPy, Pandas, and Scikit-Learn.

- Explain the preprocessing steps required before model training.
- Identify the dependent and independent variables.
- Compare two suitable machine learning algorithms for prediction.
- Discuss how SciPy and NumPy can assist in feature analysis.
- Explain how model performance should be evaluated.

The question as given (Sample Questions/6267292660481594246.jpg)

Answer, point by point

1. What preprocessing steps are required before model training?

- **Quality checks**: missing values (0), duplicate rows (0), data types (all integers). Fix or remove what is found.
- **Outliers**: no value has |z| > 2 here. Decide case by case whether to keep, cap or remove.
- **Features**: the three features are strongly correlated with each other (variance inflation factors 38.8, 10.3 and 34.1; above 10 signals trouble). Keep fewer features, or use a regularised model (Ridge) or PCA.
- **Scaling** (`StandardScaler`) for scale-sensitive models such as linear models with regularisation, k-NN and SVM; trees do not need it. Fit the scaler on training data only, inside a `Pipeline`.
- **Split** into training and test data (for example 80/20), or use cross-validation. With only five rows a test set would hold one house, so the code uses leave-one-out cross-validation.
- No categorical columns here; if there were, encode them (one-hot).

2. Which are the dependent and independent variables?

Dependent (target): `Price (Lakhs)`. **Independent (features):** `HouseSize (sq.ft.)`, `Bedrooms`, `Age (Years)`. The model learns Price = f(HouseSize, Bedrooms, Age).

3. Compare two suitable machine learning algorithms

	Linear regression	Random forest regressor
Idea	a straight-line (plane) relationship	many decision trees, predictions averaged
Interpretability	coefficients read directly (lakh per sq.ft)	feature importances only
Data needed	works with little data	needs much more data
Outside the training range	extrapolates (3000 sq.ft -> 135.0 lakh)	cannot: stays near the largest price seen (3000 sq.ft -> 103.7 lakh)
Non-linear effects, interactions	not captured	captured
Leave-one-out result here	MAE 0.00, RMSE 0.00, R-squared 1.000	MAE 9.95, RMSE 13.95, R-squared 0.643

Watch out. Do not read too much into the numbers. This table is exactly linear (Price = 0.05 x HouseSize - 15), so linear regression is perfect and Pearson r is 1.000. Real data never behaves like that, and five rows prove very little. The baseline that always predicts the mean price scores R-squared -0.562, which is the bar any model must clear.

Conclusion: start with linear regression as the interpretable baseline; move to a forest or gradient boosting when more data shows curved relationships, and keep whichever has the lower cross-validated error.

4. How can SciPy and NumPy assist in feature analysis?

- **NumPy**: `np.corrcoef`, `np.polyfit(size, price, 1)` (slope 0.05 lakh per sq.ft, intercept -15), `np.linalg.inv` of the correlation matrix to obtain the variance inflation factors.

- **SciPy**: `stats.pearsonr` (correlation and p-value), `stats.zscore` for outliers, `stats.linregress`, `stats.spearmanr` for monotonic but non-linear relations, hypothesis tests such as `ttest_ind` when groups are compared.
- Result here: size~price r = 1.000, age~price r = -0.985, bedrooms~price r = 0.949; the features carry almost the same information.

5. How should model performance be evaluated?

- **Regression metrics**: MAE (average error in lakhs), RMSE (penalises big errors), R-squared (share of variance explained). Compute them on data the model has not seen.
- **Cross-validation** (k-fold; leave-one-out for tiny data) instead of one lucky split; compare against a **baseline** (the mean price).
- Plot **residuals** against predictions: a pattern means the model is missing structure.
- Compare training error with validation error to spot over-fitting, and judge the error against the business tolerance (a 10-lakh error on a 45-lakh house is large).

Code, run on the dataset of the question

Code: code/sa_m2_q4.py

```
"""Sample answer M2-Q4: a house-price model with NumPy, pandas, SciPy and scikit-learn."""
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import LeaveOneOut, cross_val_predict
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({"HouseSize": [1200, 1500, 1800, 2200, 2500], "Bedrooms": [2, 3, 3, 4, 4],
                   "Age": [10, 8, 5, 3, 2], "Price": [45, 60, 75, 95, 110]})

print("--- preprocessing checks ---")
print("missing values:", int(df.isna().sum().sum()), "| duplicate rows:", int(df.duplicated().sum()), "| types:",
    sorted(set(df.dtypes.astype(str))))
print("values with |z| > 2:", int((np.abs(stats.zscore(df)) > 2).sum()))
X = df[["HouseSize", "Bedrooms", "Age"]]          # independent variables (features)
y = df["Price"]                                    # dependent variable (target), in lakhs
print("X:", X.shape, "y:", y.shape)

print("\n--- feature analysis with NumPy and SciPy ---")
print(df.corr().round(3))
r, p = stats.pearsonr(df["HouseSize"], df["Price"])
print("Pearson r (size, price) =", round(r, 4), "| p-value =", round(p, 5))
slope, intercept = np.polyfit(df["HouseSize"], df["Price"], 1)
print("price ~ size: slope", round(slope, 4), "lakh per sq.ft, intercept", round(intercept, 2))
vif = np.diag(np.linalg.inv(np.corrcoef(X.values, rowvar=False)))
print("variance inflation factors (above 10 means strong collinearity):", dict(zip(X.columns, vif.round(1))))

print("\n--- two algorithms and a baseline, leave-one-out cross-validation (5 rows only) ---")
models = {"baseline (mean price)": DummyRegressor(),
          "linear regression": make_pipeline(StandardScaler(), LinearRegression()),
          "random forest": RandomForestRegressor(n_estimators=200, random_state=0)}
for name, model in models.items():
    pred = cross_val_predict(model, X, y, cv=LeaveOneOut())
    print(f"{name:<22} MAE {mean_absolute_error(y, pred):6.2f}  RMSE {mean_squared_error(y, pred) ** 0.5:6.2f}  R2
        {r2_score(y, pred):7.3f}")

print("\n--- predictions after fitting on all five rows ---")
new = pd.DataFrame({"HouseSize": [2000, 3000], "Bedrooms": [3, 5], "Age": [6, 1]})
for name in ("linear regression", "random forest"):
    fitted = models[name].fit(X, y)
    print(f"{name:<18} 2000 sq.ft -> {fitted.predict(new)[0]:6.1f} lakh | 3000 sq.ft (outside the training range) ->
         {fitted.predict(new)[1]:6.1f} lakh")
```

Real output

```
--- preprocessing checks ---
missing values: 0 | duplicate rows: 0 | types: ['int64']
values with |z| > 2: 0
X: (5, 3) y: (5,)

--- feature analysis with NumPy and SciPy ---
           HouseSize  Bedrooms    Age  Price
HouseSize      1.000     0.949 -0.985  1.000
Bedrooms       0.949     1.000 -0.942  0.949
Age           -0.985    -0.942  1.000 -0.985
Price          1.000     0.949 -0.985  1.000
Pearson r (size, price) = 1.0 | p-value = 0.0
price ~ size: slope 0.05 lakh per sq.ft, intercept -15.0
variance inflation factors (above 10 means strong collinearity): {'HouseSize': 38.8, 'Bedrooms': 10.3, 'Age': 34.1}

--- two algorithms and a baseline, leave-one-out cross-validation (5 rows only) ---
baseline (mean price)  MAE  25.50  RMSE  29.21  R2  -0.562
linear regression      MAE   0.00  RMSE   0.00  R2   1.000
random forest          MAE   9.95  RMSE  13.95  R2   0.643

--- predictions after fitting on all five rows ---
linear regression  2000 sq.ft ->   85.0 lakh | 3000 sq.ft (outside the training range) ->  135.0 lakh
random forest      2000 sq.ft ->   74.2 lakh | 3000 sq.ft (outside the training range) ->  103.7 lakh
```

M2-Q5 Smart City Analytics Dashboard

Question 5: Smart City Analytics Dashboard

A municipality maintains environmental data collected from different city zones.

Dataset

Zone	Temperature (°C)	AQI	Rainfall (mm)	Green Cover (%)
North	35	180	20	18
South	30	120	35	30
East	33	150	25	22
West	29	100	40	35
Central	37	220	15	12

HOT Question

As a Smart City Data Scientist:

- Determine which environmental factors require immediate attention.
- Recommend suitable visualizations using Matplotlib, Seaborn, Plotly, Bokeh and Folium.
- Explain how interactive dashboards can support government decision-making.
- Suggest machine learning applications that could be developed using this dataset.
- Justify which visualization library is most appropriate for static reports, interactive analytics, and geographical mapping.

The question as given (Sample Questions/6267292660481594247.jpg)

Answer, point by point

1. Which environmental factors require immediate attention?

Convert AQI to the official categories of the CPCB National Air Quality Index (0-50 good, 51-100 satisfactory, 101-200 moderate, 201-300 poor, 301-400 very poor, 401+ severe) and rank the zones on all four factors together.

Zone	AQI	AQI category	Temp	Rain	Green	Priority score
Central	220	**Poor**	37	15	12	5.63
North	180	Moderate	35	20	18	2.75
East	150	Moderate	33	25	22	0.36
South	120	Moderate	30	35	30	-3.40
West	100	Satisfactory	29	40	35	-5.34

- **Central** needs attention first: the only zone in the "poor" band, the hottest (37 C), the driest (15 mm) and with the least green cover (12%). **North** is next.
- The priority score adds the z-scores of AQI and temperature and subtracts those of green cover and rainfall.
- AQI correlates +0.99 with temperature and -0.99 with green cover, -0.98 with rainfall. With five zones this shows a pattern worth investigating, not a cause.

2. Recommend suitable visualizations using Matplotlib, Seaborn, Plotly, Bokeh and Folium

Library	Chart for this data	Why
Matplotlib	bar chart of AQI by zone with a line at 200 (the "poor" limit)	full control, publication quality, static
Seaborn	correlation heat map of the four factors; pair plot when there are more zones	statistical plots in one line on top of Matplotlib
Plotly	scatter of AQI against green cover, bubble size = temperature, hover shows rainfall	interactive (hover, zoom) with almost no code, exports HTML
Bokeh	linked scatter plots with hover tools, widgets, or a live dashboard served by Bokeh server	fine-grained interactivity and streaming
Folium	map with one circle per zone, colour = AQI category, size = AQI	geographic view on Leaflet; needs zone coordinates or boundaries (not in the table, the code uses made-up coordinates)

3. How can interactive dashboards support government decision-making?

- **One live view** of all zones: officials filter by zone or time, hover for exact values, drill from city to zone.
- **Alerts**: a zone turns red when AQI passes 200, so action does not wait for a monthly report.
- **Prioritising money and work**: tree planting where green cover and rainfall are low, traffic control where AQI is high, compared across zones on the same screen.

- **Scenario thinking and transparency**: what happens to AQI if green cover rises; the same dashboard can be shown to the public.
- Limits: needs a reliable data feed, access control, and a clear owner; a dashboard supports a decision, it does not make it.

4. Which machine learning applications could be developed from this dataset?

- **Forecasting** AQI from temperature, rainfall and green cover (regression), or AQI over time (time series).
- **Classification** of a zone into the AQI category (decision tree) so that the cause of "poor" can be read from the rules.
- **Clustering** (k-means) of zones into risk groups with a common remedy; **anomaly detection** for sensor faults and sudden spikes.
- **What-if modelling**: the effect of extra green cover on AQI. All of these need far more than five rows (months of readings per zone).

5. Justify which library is most appropriate for static reports, interactive analytics and geographic mapping

Purpose	Library	Justification
Static reports	**Matplotlib** (with Seaborn)	fixed images for PDFs and papers, exact control of every element, Seaborn adds statistical plots
Interactive analytics	**Plotly** (Bokeh when you need custom linked views or a server)	hover, zoom, filtering and sharing as one HTML file with little code
Geographic mapping	**Folium**	builds Leaflet maps: markers, circles, choropleths and heat maps over real map tiles

Code, run on the dataset of the question

Code: code/sa_m2_q5.py

```
"""Sample answer M2-Q5: priority zones and dashboard building blocks for the smart-city table."""
import os
import tempfile
from pathlib import Path

import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import folium
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.plotting import figure, output_file, save

OUT = Path.cwd()                                    # the PNG goes here, the HTML dashboards into a temporary folder
os.chdir(tempfile.mkdtemp(prefix="sa_city_"))

df = pd.DataFrame({"Zone": ["North", "South", "East", "West", "Central"], "Temperature": [35, 30, 33, 29, 37],
                   "AQI": [180, 120, 150, 100, 220], "Rainfall": [20, 35, 25, 40, 15], "GreenCover": [18, 30, 22,
                       35, 12]})

print("--- 1. which factors need immediate attention? ---")
df["AQI_category"] = pd.cut(df["AQI"], bins=[0, 50, 100, 200, 300, 400, 500],
                            labels=["Good", "Satisfactory", "Moderate", "Poor", "Very Poor", "Severe"])
print(df[["Zone", "AQI", "AQI_category"]])
num = ["Temperature", "AQI", "Rainfall", "GreenCover"]
print(df[num].corr().round(2))
z = (df[num] - df[num].mean()) / df[num].std(ddof=0)
df["priority_score"] = (z["AQI"] + z["Temperature"] - z["GreenCover"] - z["Rainfall"]).round(2)    # high AQI/heat,
    low green/rain
print(df.sort_values("priority_score", ascending=False)[["Zone", "priority_score"]].to_string(index=False))

print("\n--- 2. static charts (Matplotlib + Seaborn) ---")
fig, ax = plt.subplots(1, 2, figsize=(10, 3.6))
colors = df["AQI"].map(lambda v: "#c0392b" if v > 200 else "#e69f00" if v > 100 else "#2e8b57")
ax[0].bar(df["Zone"], df["AQI"], color=colors)
ax[0].axhline(200, color="grey", linestyle="--")
ax[0].text(4.45, 203, "Poor above 200", ha="right", fontsize=8)
ax[0].set_ylabel("AQI")
ax[0].set_title("AQI by zone")
sns.heatmap(df[num].corr(), annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1, ax=ax[1])
ax[1].set_title("Correlation of the four factors")
plt.tight_layout()
plt.savefig(OUT / "sa_m2_q5.png", dpi=110)
print("saved sa_m2_q5.png")

print("\n--- 3. interactive pieces (Plotly, Bokeh) and a map (Folium) ---")
fig2 = px.scatter(df, x="GreenCover", y="AQI", size="Temperature", color="Zone", hover_data=["Rainfall"], title="AQI
    against green cover")
fig2.write_html("dashboard_plotly.html", include_plotlyjs="cdn")
src = ColumnDataSource(df)
p = figure(title="AQI against rainfall", x_axis_label="Rainfall (mm)", y_axis_label="AQI", width=420, height=300,
    tools="pan,wheel_zoom,reset")
p.scatter("Rainfall", "AQI", source=src, size=14)
p.add_tools(HoverTool(tooltips=[("Zone", "@Zone"), ("AQI", "@AQI"), ("Green cover", "@GreenCover%")]))
output_file("dashboard_bokeh.html")
save(p)
coords = {"North": (13.12, 80.25), "South": (12.93, 80.24), "East": (13.03, 80.31), "West": (13.04, 80.17),
    "Central": (13.04, 80.24)}   # illustrative
m = folium.Map(location=[13.03, 80.24], zoom_start=11)
for _, r in df.iterrows():
    colour = "red" if r["AQI"] > 200 else "orange" if r["AQI"] > 100 else "green"
    folium.CircleMarker(coords[r["Zone"]], radius=r["AQI"] / 12, color=colour, fill=True,
                        tooltip=f"{r['Zone']}: AQI {r['AQI']} ({r['AQI_category']})").add_to(m)
m.save("dashboard_map.html")
print("written:", sorted(f for f in os.listdir(".") if f.endswith(".html")))
```

Real output

```
--- 1. which factors need immediate attention? ---
      Zone  AQI  AQI_category
0    North  180      Moderate
1    South  120      Moderate
2     East  150      Moderate
3     West  100  Satisfactory
4  Central  220          Poor
             Temperature   AQI  Rainfall  GreenCover
Temperature         1.00  0.99     -0.99       -0.99
AQI                 0.99  1.00     -0.98       -0.99
Rainfall           -0.99 -0.98      1.00        1.00
GreenCover         -0.99 -0.99      1.00        1.00
   Zone  priority_score
Central            5.63
  North            2.75
   East            0.36
  South           -3.40
   West           -5.34

--- 2. static charts (Matplotlib + Seaborn) ---
saved sa_m2_q5.png

--- 3. interactive pieces (Plotly, Bokeh) and a map (Folium) ---
written: ['dashboard_bokeh.html', 'dashboard_map.html', 'dashboard_plotly.html']
```

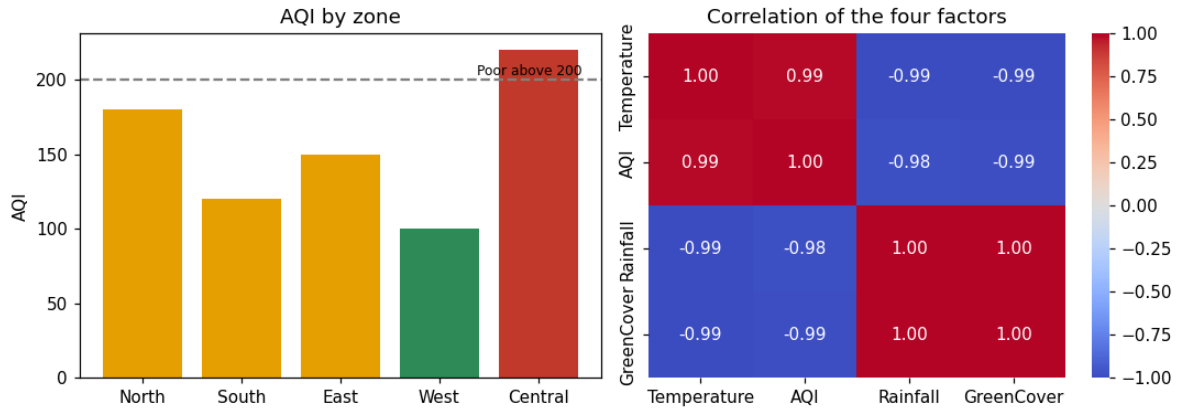


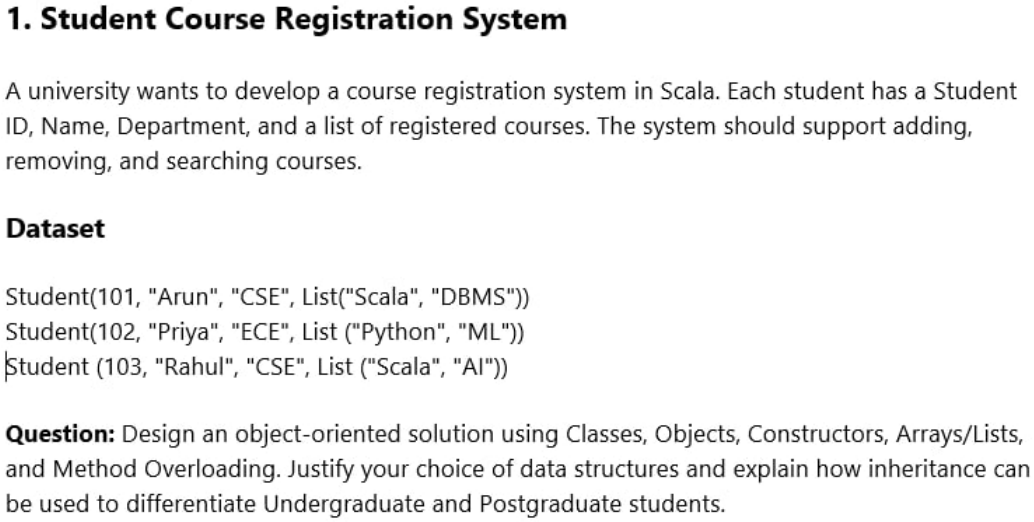
Figure produced by the code: AQI by zone with the "poor" limit (left), correlation of the four factors (right).

Note. The code also writes a Plotly page, a Bokeh page and a Folium map as HTML files (listed in the output). Their zone coordinates are invented for the demonstration.

Module 3: Scala data structures and object-oriented programming

5 sample questions with model answers.

M3-Q1 Student Course Registration System



1. Student Course Registration System

A university wants to develop a course registration system in Scala. Each student has a Student ID, Name, Department, and a list of registered courses. The system should support adding, removing, and searching courses.

Dataset

Student(101, "Arun", "CSE", List("Scala", "DBMS"))
Student(102, "Priya", "ECE", List ("Python", "ML"))
Student (103, "Rahul", "CSE", List ("Scala", "AI"))

Question: Design an object-oriented solution using Classes, Objects, Constructors, Arrays/Lists, and Method Overloading. Justify your choice of data structures and explain how inheritance can be used to differentiate Undergraduate and Postgraduate students.

The question as given (Sample Questions/6271407831856650886.jpg)

Answer

Design in one picture

Part	Scala feature	Role
`abstract class Student(id, name, dept, initialCourses)`	class, primary constructor, abstract members	common fields and the add / remove / search rules
`UGStudent`, `PGStudent`	inheritance, auxiliary constructor, default argument	supply `maxCourses` and `level`; PG adds `thesisTopic`
`addCourse(String)` and `addCourse(List[String])`	method overloading	add one course or several at once
`Registry.find(Int)` and `Registry.find(String)`	singleton object, overloading	search by id or by name
`private var registered: List[String]`	encapsulation	outsiders get the list through `courses`, they cannot change it

Why this choice of data structure?

Structure	Fits this problem?	Reason
`Array[String]`	no	fixed size: adding or removing a course means building a new array; elements can be changed from outside
`List[String]` **(chosen)**	**yes**	a student has at most 4 to 6 courses, so the O(n) append does not matter; it is immutable, so it can be handed out safely and gives the clean copy semantics of functional code
`ListBuffer[String]`	yes, for heavy editing	fast in-place add and remove, but mutable state leaks if exposed
`Set[String]`	partly	removes duplicates automatically but loses the order of registration; here `contains` plus `distinct` do the same
`Map[Int, Student]` for the registry	yes at scale	O(1) lookup by id; the demo uses a `ListBuffer` scan because there are three students

How does inheritance differentiate Undergraduate and Postgraduate students?

- The base class holds everything that is the same (fields, `addCourse`, `removeCourse`, `hasCourse`) and declares what differs as **abstract** members: `maxCourses` and `level`.

- `UGStudent` says `maxCourses = 6`, `level = "UG"`. `PGStudent` says `maxCourses = 4`, `level = "PG"` and adds a `thesisTopic`.
- **Polymorphism**: the same call `student.addCourse(...)` obeys the limit of the real class. In the output Arun (UG) accepts 3 of 4 new courses because he reaches 6, Priya (PG) accepts 2 of 3 because she reaches 4.
- `maxCourses` is a `def`, not a `val`: a parent constructor that reads an overridden `val` sees 0 (initialisation-order trap), a `def` is always safe.

Note. The question does not say which student is UG or PG. The code treats Arun and Rahul as undergraduates and Priya as a postgraduate.

Code, run on the dataset of the question

Code: code/sa_m3_q1.scala

```
//> using scala 2.13.12
// Sample answer M3-Q1: course registration system (classes, objects, constructors, lists, method overloading,
    inheritance)
import scala.collection.mutable.ListBuffer

abstract class Student(val id: Int, val name: String, val dept: String, initialCourses: List[String]) {
  private var registered: List[String] = initialCourses.distinct   // an immutable List behind a private var

  def maxCourses: Int                                              // the rule comes from the subclass (def, not
      val)
  def level: String

  def courses: List[String] = registered                           // callers receive the immutable list, they
      cannot change ours
  def hasCourse(course: String): Boolean = registered.contains(course)

  def addCourse(course: String): Boolean =                         // overloaded: one course
    if (hasCourse(course) || registered.size >= maxCourses) false
    else { registered = registered :+ course; true }

  def addCourse(newCourses: List[String]): Int =                   // overloaded: several courses, returns how many
      were added
    newCourses.count(c => addCourse(c))

  def removeCourse(course: String): Boolean =
    if (hasCourse(course)) { registered = registered.filterNot(_ == course); true } else false
}

class UGStudent(id: Int, name: String, dept: String, initial: List[String]) extends Student(id, name, dept, initial)
     {
  def this(id: Int, name: String, dept: String) = this(id, name, dept, Nil)    // auxiliary constructor
  def maxCourses = 6
  def level = "UG"
}

class PGStudent(id: Int, name: String, dept: String, initial: List[String], val thesisTopic: String = "to be
    decided")
    extends Student(id, name, dept, initial) {
  def maxCourses = 4
  def level = "PG"
}

object Registry {                                                  // singleton: one registry for the whole
    application
  private val students = ListBuffer[Student]()
  def register(s: Student): Unit = students += s
  def find(id: Int): Option[Student] = students.find(_.id == id)                       // overloaded search: by id
  def find(name: String): Option[Student] = students.find(_.name.equalsIgnoreCase(name)) // overloaded search: by
      name
  def enrolledIn(course: String): List[String] = students.filter(_.hasCourse(course)).map(_.name).toList
  def report(): Unit = students.foreach { s =>
    println(s.id + " " + s.name + " (" + s.dept + ", " + s.level + ", max " + s.maxCourses + "): " +
        s.courses.mkString("[", ", ", "]"))
  }
}

object CourseRegistration {
  def main(args: Array[String]): Unit = {
    Registry.register(new UGStudent(101, "Arun", "CSE", List("Scala", "DBMS")))
    Registry.register(new PGStudent(102, "Priya", "ECE", List("Python", "ML"), "speech models"))
    Registry.register(new UGStudent(103, "Rahul", "CSE", List("Scala", "AI")))
    println("--- registered students ---")
    Registry.report()

    println("--- searching (two overloaded find methods) ---")
    println(Registry.find(101).map(_.name))
    println(Registry.find("priya").map(_.id))
    println(Registry.find(999))
    println(Registry.enrolledIn("Scala"))

    println("--- adding and removing courses ---")
    val arun = Registry.find(101).get
    println("add Statistics     : " + arun.addCourse("Statistics"))
    println("add Scala again    : " + arun.addCourse("Scala"))
    println("add four more      : " + arun.addCourse(List("OS", "CN", "Maths", "Physics")) + " added (UG limit is
        6)")
    println("remove DBMS        : " + arun.removeCourse("DBMS") + ", remove Chemistry: " +
        arun.removeCourse("Chemistry"))
    println("Arun now           : " + arun.courses)
    val priya = Registry.find(102).get
    println("Priya adds three   : " + priya.addCourse(List("Stats", "NLP", "Vision")) + " added (PG limit is 4)")
    println("Priya now          : " + priya.courses)
  }
}
```

Real output

```
--- registered students ---
101 Arun (CSE, UG, max 6): [Scala, DBMS]
102 Priya (ECE, PG, max 4): [Python, ML]
103 Rahul (CSE, UG, max 6): [Scala, AI]
--- searching (two overloaded find methods) ---
Some(Arun)
Some(102)
None
List(Arun, Rahul)
--- adding and removing courses ---
add Statistics     : true
add Scala again    : false
add four more      : 3 added (UG limit is 6)
remove DBMS        : true, remove Chemistry: false
Arun now           : List(Scala, Statistics, OS, CN, Maths)
Priya adds three   : 2 added (PG limit is 4)
Priya now          : List(Python, ML, Stats, NLP)
```

M3-Q2 Smart Library Management System

2. Smart Library Management System

A digital library stores different types of learning resources such as Books, Journals, and Magazines.

Dataset

Book(101, "Scala Programming", "Available")
Journal(201, "Data Science Journal", "Issued")
Magazine(301, "Tech Today", "Available")

Question: Develop a class hierarchy using Inheritance, Abstract Classes, and Field Overriding. Explain how polymorphism can be used to issue and return different resource types. Discuss where the `final` keyword should be applied and justify your answer.

The question as given (Sample Questions/6271407831856650919.jpg)

Answer

Class hierarchy with inheritance, an abstract class and field overriding

- `abstract class LibraryResource(id, title, initialStatus)` holds what every resource has: id, title, a private `status`, and the rules `issue()` and `giveBack()`.
- It declares two abstract members that each subclass must supply: the field `kind` and the method `loanDays`.
- It also has a concrete field with a default, `lateFeePerDay = 1.0`. **Field overriding**: `Journal` replaces it with 5.0 and `Magazine` with 0.5 using `override val`.
- `Book` (14 days), `Journal` (7 days), `Magazine` (3 days) extend it. Adding a `Newspaper` later means one new small class and no change to the library code.

How does polymorphism issue and return different resource types?

The library keeps one `List[LibraryResource]` and calls `r.issue()` or `r.giveBack(2)` on each element. The compiler only knows the type `LibraryResource`, the JVM picks the real class at run time, so each object uses its own `kind`, `loanDays` and `lateFeePerDay`. In the output the same call prints "issued for 14 days" for the book and "issued for 3 days" for the magazine, and the same 2 days of delay costs Rs. 2.00, Rs. 10.00 and Rs. 1.00. The journal, which starts as "Issued" in the data, answers "is not available": its status is checked by the shared code.

Where should the `final` keyword be applied?

Place	Example	Justification
Methods that carry a business rule	`final def issue()`, `final def giveBack()`	every resource must follow the same rule; a subclass must not weaken it (for example issue an item without checking its status)
Classes not designed for extension	`final class Magazine`	prevents unexpected subclasses and lets the compiler optimise
Constants	`final val MaxRenewals = 2`	cannot be shadowed or changed by a subclass
Not on customisation points	`loanDays`, `lateFeePerDay`, the abstract members	they exist precisely so that subclasses can supply or override them

What the compiler says when the rules are broken (two small programs that are meant to fail):

Overriding a final method

```
compilation FAILED (as intended)
sa_m3_q2_final_error.scala:7:16
  cannot override final member:
  final def issue(): String (defined in class Resource)
```

Extending a final class

```
compilation FAILED (as intended)
sa_m3_q2_final_class_error.scala:6:31
  illegal inheritance from final class Magazine
```

Tip. Over-using `final` makes a library hard to extend and hard to test. Protect rules and invariants, leave behaviour open.

Code, run on the dataset of the question

Code: code/sa_m3_q2.scala

```
//> using scala 2.13.12
// Sample answer M3-Q2: library resources (inheritance, abstract class, field overriding, polymorphism, final)
abstract class LibraryResource(val id: Int, val title: String, initialStatus: String) {
  private var status: String = initialStatus        // "Available" or "Issued"
  val kind: String                                  // abstract field, every subclass gives its own value
  def loanDays: Int                                 // abstract method
  val lateFeePerDay: Double = 1.0                   // concrete field with a default that subclasses may override

  def isAvailable: Boolean = status == "Available"

  final def issue(): String =                       // final: the issuing rule is the same for every resource and
      cannot be changed
    if (isAvailable) { status = "Issued"; s"$kind '$title' issued for $loanDays days" }
    else s"$kind '$title' is not available"

  final def giveBack(daysLate: Int): String =
    if (!isAvailable) { status = "Available"; f"$kind '$title' returned, late fee Rs. ${daysLate *
        lateFeePerDay}%.2f" }
    else s"$kind '$title' was not issued"

  override def toString: String = s"$kind($id, $title, $status)"
}

class Book(id: Int, title: String, status: String) extends LibraryResource(id, title, status) {
  val kind = "Book"
  def loanDays = 14
}

class Journal(id: Int, title: String, status: String) extends LibraryResource(id, title, status) {
  val kind = "Journal"
  def loanDays = 7
  override val lateFeePerDay = 5.0                  // field overriding
}

final class Magazine(id: Int, title: String, status: String) extends LibraryResource(id, title, status) {
  val kind = "Magazine"                             // final class: nobody extends it further
  def loanDays = 3
  override val lateFeePerDay = 0.5
}

object Library {
  def main(args: Array[String]): Unit = {
    val shelf: List[LibraryResource] = List(
      new Book(101, "Scala Programming", "Available"),
      new Journal(201, "Data Science Journal", "Issued"),
      new Magazine(301, "Tech Today", "Available"))

    println("--- the catalogue ---")
    shelf.foreach(println)
    println("--- issue every resource: one call, three behaviours (polymorphism) ---")
    shelf.foreach(r => println(r.issue()))
    println("--- return every resource, 2 days late ---")
    shelf.foreach(r => println(r.giveBack(2)))
    println("--- state after the round ---")
    shelf.foreach(println)
    println(shelf.map(r => r.kind -> r.loanDays))
  }
}
```

Real output

```
--- the catalogue ---
Book(101, Scala Programming, Available)
Journal(201, Data Science Journal, Issued)
Magazine(301, Tech Today, Available)
--- issue every resource: one call, three behaviours (polymorphism) ---
Book 'Scala Programming' issued for 14 days
Journal 'Data Science Journal' is not available
Magazine 'Tech Today' issued for 3 days
--- return every resource, 2 days late ---
Book 'Scala Programming' returned, late fee Rs. 2.00
Journal 'Data Science Journal' returned, late fee Rs. 10.00
Magazine 'Tech Today' returned, late fee Rs. 1.00
--- state after the round ---
Book(101, Scala Programming, Available)
Journal(201, Data Science Journal, Available)
Magazine(301, Tech Today, Available)
List((Book,14), (Journal,7), (Magazine,3))
```

M3-Q3 Online Shopping Cart Analysis

3. Online Shopping Cart Analysis

An e-commerce platform stores customer purchases.

Dataset

```
Map(
  "Laptop" -> 65000,
  "Mobile" -> 25000,
  "Tablet" -> 18000,
  "Headphone" -> 3000
)
```

Question: Using Maps and Functional Combinators (`map`, `filter`, `reduce`), design a solution to:

- Apply a 10% discount to all products.
- Identify products costing more than ₹20,000.
- Calculate the total inventory value.

Compare an imperative solution with a functional programming solution and justify which is more scalable.

The question as given (Sample Questions/6271407831856650920.jpg)

Answer

Task	Functional combinator	Result
10% discount on all products	`prices.map { case (item, p) => item -> p * 0.9 }`	Laptop 58500, Mobile 22500, Tablet 16200, Headphone 2700
Products costing more than Rs. 20,000	`prices.filter { case (_, p) => p > 20000 }`	Laptop, Mobile
Total inventory value (one unit of each)	`prices.values.reduce(_ + _)`	111,000 (99,900 after the discount)

The three steps chain into one readable pipeline, for example `prices.filter(_._2 > 20000).map { case (i, p) => i -> p * 0.9 }.values.sum` (the discounted value of the expensive products, 81,000).

Imperative versus functional, and which is more scalable

	Imperative (loop and variables)	Functional (combinators)
State	mutable `var total`, a mutable map that is filled in place	nothing is changed; every step returns a new value
Reading	how to do it, step by step	what is wanted: map, filter, reduce
Testing	depends on the order of the statements	each step is a pure function and can be tested alone
Running in parallel	shared mutable state needs locks, otherwise race conditions	no shared state, so the work splits into chunks safely
Moving to a cluster	must be rewritten	the same `map` / `filter` / `reduce` vocabulary is the API of Spark RDDs

Justification: the functional version is more scalable. It has no shared mutable state, so the collection can be split into parts, each part processed independently and the partial results combined, which is exactly how parallel collections and Spark work. The imperative version is perfectly fine for a small local loop; it is the mutation that does not scale. Both versions give identical answers in the output (`same answers: true`).

Watch out. `reduce` has no starting value and throws on an empty collection (the real message is "empty.reduceLeft"). `foldLeft(0)(_ + _)` returns 0. Use `foldLeft` when the collection can be empty.

Note. Prices are Int and the discount is applied as `p * 0.9` (Double). For real money use `BigDecimal` and round, because Double arithmetic can give results such as 60500.00000000001.

Code, run on the dataset of the question

Code: code/sa_m3_q3.scala

```
//> using scala 2.13.12
// Sample answer M3-Q3: Map and functional combinators (map, filter, reduce) compared with an imperative loop
import scala.collection.mutable

object Cart {
  def main(args: Array[String]): Unit = {
    val prices = Map("Laptop" -> 65000, "Mobile" -> 25000, "Tablet" -> 18000, "Headphone" -> 3000)
    val show = (m: Map[String, Double]) => m.toList.sortBy(_._1).map { case (k, v) => f"$k -> $v%.1f" }.mkString(",
        ")

    println("--- functional version: no variable is ever changed ---")
    val discounted = prices.map { case (item, p) => item -> p * 0.9 }          // 10% discount on every product
    val expensive = prices.filter { case (_, p) => p > 20000 }                  // products above Rs. 20,000
    val totalValue = prices.values.reduce(_ + _)                                // total inventory value (one unit
        of each)
    println("discounted : " + show(discounted))
    println("expensive  : " + expensive.keys.toList.sorted)
    println("total      : " + totalValue + " (after discount " + discounted.values.sum + ")")

    println("--- imperative version of the same three tasks ---")
    val discounted2 = mutable.Map[String, Double]()
    var expensive2 = List[String]()
    var total2 = 0
    for ((item, p) <- prices) {
      discounted2(item) = p * 0.9
      if (p > 20000) expensive2 = item :: expensive2
      total2 += p
    }
    println("discounted : " + show(discounted2.toMap))
    println("expensive  : " + expensive2.sorted)
    println("total      : " + total2)
    println("same answers: " + (discounted == discounted2.toMap && expensive.keySet == expensive2.toSet &&
        totalValue == total2))

    println("--- a chained pipeline reads like the requirement ---")
    val report = prices.filter(_._2 > 20000).map { case (i, p) => i -> p * 0.9 }.values.sum
    println("discounted value of the expensive products: " + report)

    println("--- reduce fails on an empty collection, foldLeft does not ---")
    try Map.empty[String, Int].values.reduce(_ + _)
    catch { case e: UnsupportedOperationException => println("reduce: " + e.getMessage) }
    println("foldLeft: " + Map.empty[String, Int].values.foldLeft(0)(_ + _))
  }
}
```

Real output

```
--- functional version: no variable is ever changed ---
discounted : Headphone -> 2700.0, Laptop -> 58500.0, Mobile -> 22500.0, Tablet -> 16200.0
expensive  : List(Laptop, Mobile)
total      : 111000 (after discount 99900.0)
--- imperative version of the same three tasks ---
discounted : Headphone -> 2700.0, Laptop -> 58500.0, Mobile -> 22500.0, Tablet -> 16200.0
expensive  : List(Laptop, Mobile)
total      : 111000
same answers: true
--- a chained pipeline reads like the requirement ---
discounted value of the expensive products: 81000.0
--- reduce fails on an empty collection, foldLeft does not ---
reduce: empty.reduceLeft
foldLeft: 0
```

M3-Q4 Employee Management using Case Classes and Companion Objects

4. Employee Management using Case Classes and Companion Objects

A company stores employee information.

Dataset

Employee(101, "Ravi", "Developer", 75000)
Employee(102, "Anu", "Tester", 55000)
Employee(103, "Kiran", "Manager", 95000)

Question: Design the Employee system using Case Classes and Companion Objects. Explain:

- Why case classes are suitable for employee records.
- How companion objects can be used to create utility methods.
- How access modifiers improve data security.
- How Singleton Objects can be used to generate employee reports.

The question as given (Sample Questions/6271407831856650921.jpg)

Answer, point by point

Why are case classes suitable for employee records?

- An employee record is a **value**: defined by its fields, not by identity. A case class gives value semantics for free: `equals` / `hashCode` compare the fields (`Employee(101, ...) == Employee(101, ...)` is true), `toString` prints `Employee(101,Ravi,Developer,75000.0)`.
- Fields are immutable `val`s, so a record cannot change under your feet; a changed copy is made with `copy(salary = 80000)`.
- No `new` needed (generated `apply`), and the generated `unapply` allows **pattern matching** on the fields: `case Employee(_, n, "Manager", s) if s > 90000 => ...`.
- Caution: do not put mutable fields into a case class and avoid case-class inheritance.

How can companion objects create utility methods?

A companion is an `object` with the same name in the same file. It holds everything that belongs to the concept `Employee` but not to one employee: the constant `MinSalary`, the factory `fromCsv(line): Option[Employee]` that parses and validates a text line, `raise(e, percent)` and `totalPayroll(list)`. In the output `fromCsv` accepts a good line and returns `None` for a salary below the minimum and for malformed lines. The class and its companion can see each other's private members.

How do access modifiers improve data security?

Modifier	Visible to	Use in this design
(none) = public	everyone	the case-class fields and the public methods
`private`	only the enclosing class or object (and its companion)	`taxRate` and `net()` inside `PayrollReport`: the tax rule cannot be read or called from outside
`protected`	the class and its subclasses	helper methods meant for subclasses
`private[this]`	only the same instance	per-object internal state

Hiding the details means other code can only use the safe public methods, so rules (validation, rounding, tax) live in one place. Trying to touch a private member is a compile error, not a run-time surprise (this file is meant to fail):

Reading a private member from outside

```
compilation FAILED (as intended)
sa_m3_q4_private_error.scala:12:13
  value taxRate in object PayrollReport cannot be accessed as a member of object PayrollReport from object Wrong
```

How can singleton objects generate employee reports?

`object PayrollReport` has exactly one instance, created lazily and thread-safely by the JVM, and is called as `PayrollReport.generate(staff)` without `new`. It is the right shape for a stateless service such as a report generator or a utility: no object has to be passed around, and its private helpers stay hidden. The output shows the gross and net salary of each employee, the total payroll (225,000), the average (75,000) and the highest paid employee (Kiran).

Code, run on the dataset of the question

Code: code/sa_m3_q4.scala

```
//> using scala 2.13.12
// Sample answer M3-Q4: employees with case classes, a companion object, access modifiers and a singleton report
    generator
case class Employee(id: Int, name: String, role: String, salary: Double)

object Employee {                                   // companion: same name, same file; holds the utility methods
  val MinSalary = 30000.0
  def fromCsv(line: String): Option[Employee] = line.split(",").map(_.trim) match {
    case Array(id, name, role, salary) =>
      scala.util.Try(Employee(id.toInt, name, role, salary.toDouble)).toOption.filter(_.salary >= MinSalary)
    case _ => None
  }
  def raise(e: Employee, percent: Double): Employee = {          // money: round to paise, 55000 * 1.1 alone gives
      60500.00000000001
    val raised = BigDecimal(e.salary * (1 + percent / 100)).setScale(2, BigDecimal.RoundingMode.HALF_UP).toDouble
    e.copy(salary = raised)
  }
  def totalPayroll(es: Seq[Employee]): Double = es.map(_.salary).sum
}

object PayrollReport {                              // singleton object: the one and only report generator
  private val taxRate = 0.10                        // private: not visible outside this object
  private def net(e: Employee): Double = e.salary * (1 - taxRate)
  def generate(es: List[Employee]): String = {
    val lines = es.map(e => f"${e.id}%d ${e.name}%-5s ${e.role}%-9s gross ${e.salary}%9.1f net ${net(e)}%9.1f")
    val top = es.maxBy(_.salary)
    (lines :+ f"payroll ${Employee.totalPayroll(es)}%.1f, average ${Employee.totalPayroll(es) / es.size}%.1f,
        highest paid ${top.name}").mkString("\n")
  }
}

object EmployeeApp {
  def main(args: Array[String]): Unit = {
    val staff = List(Employee(101, "Ravi", "Developer", 75000), Employee(102, "Anu", "Tester", 55000), Employee(103,
         "Kiran", "Manager", 95000))

    println("--- what a case class gives for free ---")
    println(staff.head)                                                  // readable toString
    println(Employee(101, "Ravi", "Developer", 75000) == staff.head)    // equality by value
    println(staff.head.copy(salary = 80000))                             // immutable update
    staff(2) match {                                                     // pattern matching on the fields
      case Employee(_, n, "Manager", s) if s > 90000 => println(n + " is a senior manager")
      case _ => println("other")
    }

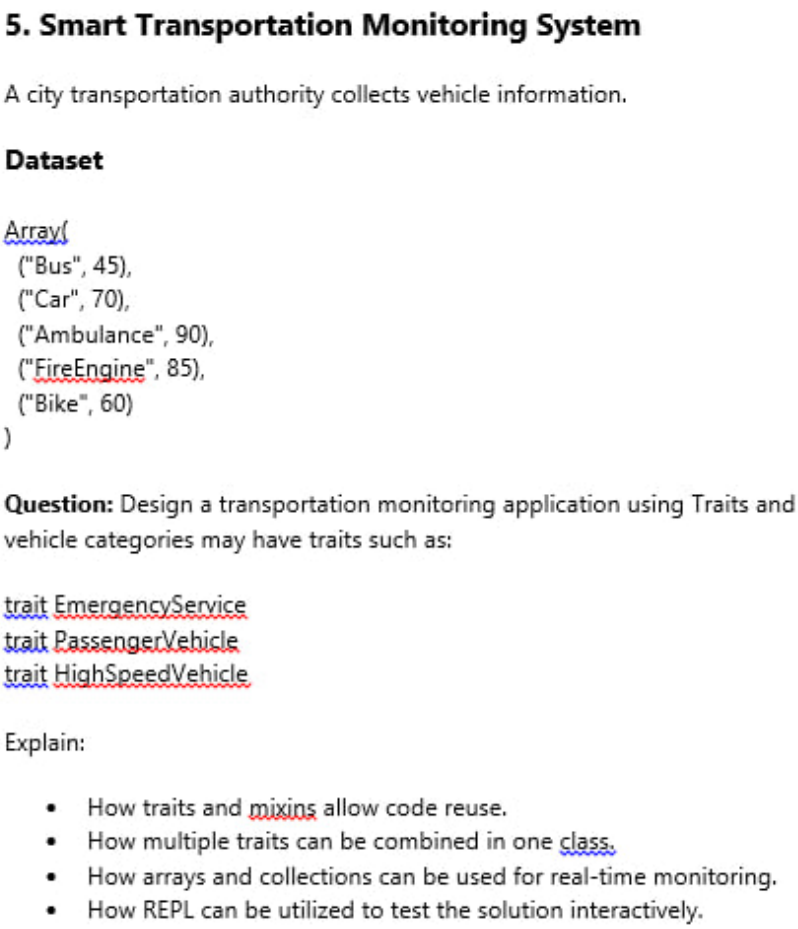
    println("--- companion object utilities ---")
    println(Employee.fromCsv("104, Meena, Analyst, 62000"))
    println(Employee.fromCsv("105, Bad, Intern, 12000") + " (below the minimum of " + Employee.MinSalary + ")")
    println(Employee.fromCsv("106, Oops, Tester") + " and " + Employee.fromCsv("x, Y, Z, abc") + " (malformed lines
        are rejected)")
    println(Employee.raise(staff(1), 10))

    println("--- singleton report ---")
    println(PayrollReport.generate(staff))
  }
}
```

Real output

```
--- what a case class gives for free ---
Employee(101,Ravi,Developer,75000.0)
true
Employee(101,Ravi,Developer,80000.0)
Kiran is a senior manager
--- companion object utilities ---
Some(Employee(104,Meena,Analyst,62000.0))
None (below the minimum of 30000.0)
None and None (malformed lines are rejected)
Employee(102,Anu,Tester,60500.0)
--- singleton report ---
101 Ravi  Developer gross   75000.0 net   67500.0
102 Anu   Tester    gross   55000.0 net   49500.0
103 Kiran Manager   gross   95000.0 net   85500.0
payroll 225000.0, average 75000.0, highest paid Kiran
```

M3-Q5 Smart Transportation Monitoring System



5. Smart Transportation Monitoring System

A city transportation authority collects vehicle information.

Dataset

```
Array(
  ("Bus", 45),
  ("Car", 70),
  ("Ambulance", 90),
  ("FireEngine", 85),
  ("Bike", 60)
)
```

Question: Design a transportation monitoring application using Traits and Trait Mixins. Different vehicle categories may have traits such as:

```
trait EmergencyService
trait PassengerVehicle
trait HighSpeedVehicle
```

Explain:

- How traits and mixins allow code reuse.
- How multiple traits can be combined in one class.
- How arrays and collections can be used for real-time monitoring.
- How REPL can be utilized to test the solution interactively.

The question as given (Sample Questions/6271407831856650922.jpg)

Answer, point by point

How do traits and mixins allow code reuse?

A trait packages behaviour (methods, fields, abstract members) that unrelated classes can reuse without sharing a parent class. `EmergencyService` (the `siren`), `PassengerVehicle` (`seats`, `boarding`) and `HighSpeedVehicle` (a higher speed `limit`) are written once and attached to the classes that need them, instead of being copied into `Ambulance`, `FireEngine`, `Bus` and `Car`. A mixin is such an attachment: `class Ambulance(...) extends Vehicle(...) with EmergencyService with HighSpeedVehicle`, or at creation time: `new Car(80) with EmergencyService with HighSpeedVehicle`.

How can multiple traits be combined in one class?

- List them after the superclass with `with`: `extends Vehicle("Ambulance", speed) with EmergencyService with HighSpeedVehicle`. A class has one parent class but any number of traits.
- When two traits define the same member, **linearisation** decides: the trait mixed in last wins. `HighSpeedVehicle` overrides `limit` (60 becomes 100), so Ambulance at 90 km/h is "ok" while Car at 70 km/h is "OVERSPEED".
- Traits in Scala 2 have no constructor parameters; traits can extend a class (here `Vehicle`), which restricts them to subclasses of it.
- Abstract class or trait? Use a trait for a reusable capability, an abstract class for the one "is-a" parent (`Vehicle`).

How can arrays and collections be used for real-time monitoring?

- The question's `Array(("Bus", 45), ...)` is turned into an `Array[Vehicle]` with one `map` and a small factory (`build`).
- Monitoring is then collection code: `filter(_.status == "OVERSPEED")`, `maxBy(_.speed)`, average speed, `collect { case v: EmergencyService => v.name }` to find the emergency vehicles. In the output only the car is overspeeding, the ambulance is the fastest, the average is 70.0 km/h.
- For a live feed the array is replaced each second (or the logic moves to Spark Streaming); the monitoring functions stay the same because they are written against `Vehicle`.

How can the REPL be used to test the solution interactively?

Start it with `scala` (or `scala-cli repl`), type definitions and expressions and read the type and value immediately; `:quit` leaves. Below is a real session (the lines after each `scala>` prompt were typed, the others are the REPL's answers). It checks the trait in isolation, mixes it into one object, and tries the array logic before it goes into the program. Unnamed results are called `res0`, `res1`, ... and an assignment such as `fleet(0) = ...` also uses up a number although it prints nothing.

A Scala REPL session (real transcript)

```
scala> trait EmergencyService { def siren: String = "siren ON" }
trait EmergencyService
scala> class Car(val speed: Int) { override def toString = "Car(" + speed + ")" }
class Car
scala> val police = new Car(80) with EmergencyService
val police: Car with EmergencyService = Car(80)
scala> police.siren
val res0: String = siren ON
scala> police.isInstanceOf[EmergencyService]
val res1: Boolean = true
scala> val fleet = Array(("Bus", 45), ("Car", 70), ("Ambulance", 90))
val fleet: Array[(String, Int)] = Array((Bus,45), (Car,70), (Ambulance,90))
scala> fleet.map(_._2).max
val res2: Int = 90
scala> fleet.filter(_._2 > 50).toList
val res3: List[(String, Int)] = List((Car,70), (Ambulance,90))
scala> fleet(0) = ("Bus", 55)
scala> fleet.map(_._2).sum
val res5: Int = 215
```

Code, run on the dataset of the question

Code: code/sa_m3_q5.scala

```scala
//> using scala 2.13.12
// Sample answer M3-Q5: vehicle monitoring with traits and mixins, driven by an Array of (name, speed) pairs
abstract class Vehicle(val name: String, val speed: Int) {
  def limit: Int = 60                              // city speed limit, km/h
  def status: String = if (speed > limit) "OVERSPEED" else "ok"
}

trait PassengerVehicle {                           // reusable behaviour no. 1
  def seats: Int
  def boarding: String = s"$seats seats"
}

trait EmergencyService extends Vehicle {           // reusable behaviour no. 2
  def siren: String = s"$name: siren ON, right of way"
}

trait HighSpeedVehicle extends Vehicle {           // reusable behaviour no. 3: overrides the limit
  override def limit: Int = 100
}

class Bus(speed: Int) extends Vehicle("Bus", speed) with PassengerVehicle { def seats = 50 }
class Car(speed: Int) extends Vehicle("Car", speed) with PassengerVehicle { def seats = 4 }
class Ambulance(speed: Int) extends Vehicle("Ambulance", speed) with EmergencyService with HighSpeedVehicle
class FireEngine(speed: Int) extends Vehicle("FireEngine", speed) with EmergencyService with HighSpeedVehicle
class Bike(speed: Int) extends Vehicle("Bike", speed)

object Monitor {
  def build(name: String, speed: Int): Vehicle = name match {
    case "Bus" => new Bus(speed)
    case "Car" => new Car(speed)
    case "Ambulance" => new Ambulance(speed)
    case "FireEngine" => new FireEngine(speed)
    case _ => new Bike(speed)
  }

  def main(args: Array[String]): Unit = {
    val data = Array(("Bus", 45), ("Car", 70), ("Ambulance", 90), ("FireEngine", 85), ("Bike", 60))
    val fleet: Array[Vehicle] = data.map { case (n, s) => build(n, s) }

    println("--- real-time check of the whole fleet ---")
    fleet.foreach(v => println(f"${v.name}%-10s ${v.speed}%3d km/h  limit ${v.limit}%3d  ${v.status}"))
    println("overspeeding: " + fleet.filter(_.status == "OVERSPEED").map(_.name).toList)
    println("fastest     : " + fleet.maxBy(_.speed).name)
    println("average     : " + fleet.map(_.speed).sum.toDouble / fleet.length)

    println("--- what each mixin adds ---")
    fleet.foreach {
      case v: EmergencyService => println(v.siren)
      case v: PassengerVehicle => println(v.name + ": " + v.boarding)
      case v => println(v.name + ": no special trait")
    }
    println("emergency vehicles: " + fleet.collect { case v: EmergencyService => v.name }.toList)

    println("--- mixing a trait into one object at creation time ---")
    val police = new Car(80) with EmergencyService with HighSpeedVehicle
    println(police.siren + " | limit " + police.limit + " | " + police.status + " | " + police.boarding)
  }
}
```

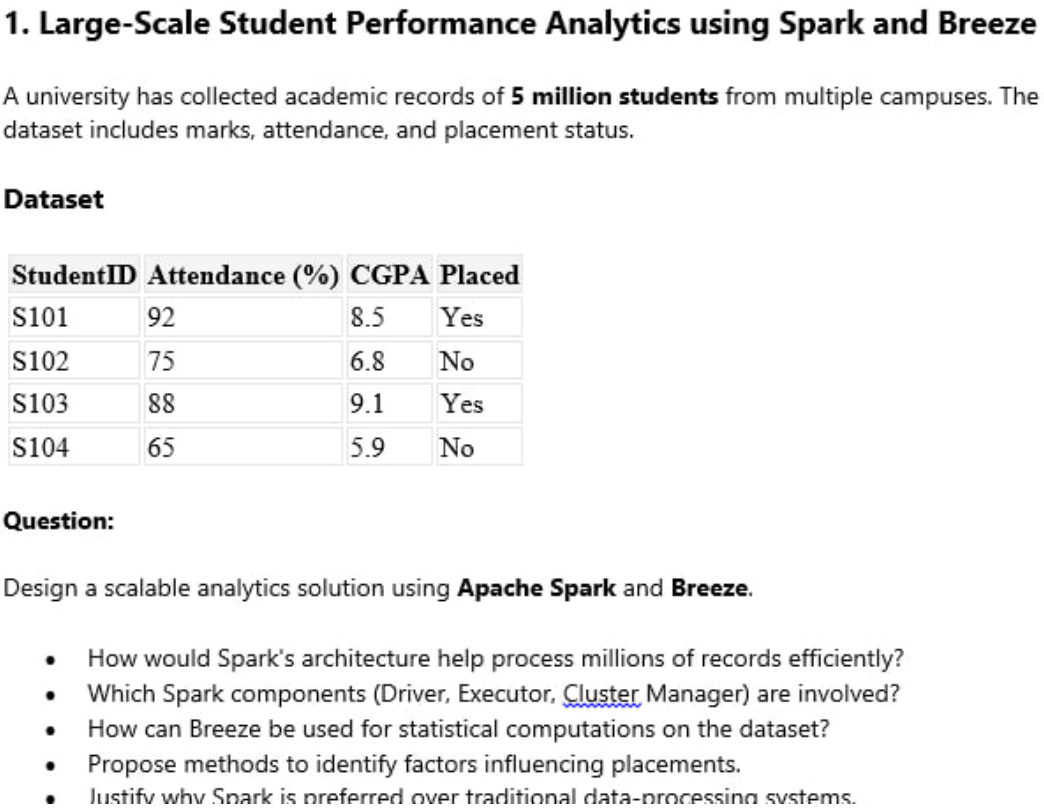
Real output

```
--- real-time check of the whole fleet ---
Bus         45 km/h  limit  60  ok
Car         70 km/h  limit  60  OVERSPEED
Ambulance   90 km/h  limit 100  ok
FireEngine  85 km/h  limit 100  ok
Bike        60 km/h  limit  60  ok
overspeeding: List(Car)
fastest     : Ambulance
average     : 70.0
--- what each mixin adds ---
Bus: 50 seats
Car: 4 seats
Ambulance: siren ON, right of way
FireEngine: siren ON, right of way
Bike: no special trait
emergency vehicles: List(Ambulance, FireEngine)
--- mixing a trait into one object at creation time ---
Car: siren ON, right of way | limit 100 | ok | 4 seats
```

Module 4: Scala libraries and Spark basics

5 sample questions with model answers.

M4-Q1 Large-Scale Student Performance Analytics using Spark and Breeze



1. Large-Scale Student Performance Analytics using Spark and Breeze

A university has collected academic records of **5 million students** from multiple campuses. The dataset includes marks, attendance, and placement status.

Dataset

StudentID	Attendance (%)	CGPA	Placed
S101	92	8.5	Yes
S102	75	6.8	No
S103	88	9.1	Yes
S104	65	5.9	No

Question:

Design a scalable analytics solution using **Apache Spark** and **Breeze**.

- How would Spark's architecture help process millions of records efficiently?
- Which Spark components (Driver, Executor, Cluster Manager) are involved?
- How can Breeze be used for statistical computations on the dataset?
- Propose methods to identify factors influencing placements.
- Justify why Spark is preferred over traditional data-processing systems.

The question as given (Sample Questions/6278527581963359885.jpg)

Answer, point by point

> **Note.** The screenshot of this question is cut off after the fifth bullet. The five visible bullets are answered; the fifth one is the last line that can be read ("Justify why Spark is preferred over traditional data-processing systems").

1. How would Spark's architecture help process millions of records efficiently?

- **Partitioning**: the 5 million rows are cut into partitions and every partition is processed by its own task, in parallel on the executors. In the demo, `spark.range(5000000)` is split into 4 partitions, so 4 tasks run at once.
- **Two-phase aggregation**: each task first aggregates its own partition (`partial_avg` in the physical plan) and only the small partial results travel through the one shuffle (`Exchange`) to the final aggregate. Little data moves.
- **Lazy evaluation and the DAG**: transformations only build a plan; the optimiser (Catalyst) rewrites the whole plan before anything runs; an action starts the job.
- **In-memory processing and caching** of reused data, **data locality** (tasks go to the node that holds the data), and **scale-out**: more executors, same code.
- **Fault tolerance**: a lost partition is recomputed from its lineage.

2. Which Spark components (Driver, Executor, Cluster Manager) are involved?

Component	What it does	In this demo
Driver	runs `main()`, owns the SparkSession, turns the code into a DAG of stages and tasks, schedules the tasks and collects the results	the JVM of the program
Cluster manager	decides which worker nodes give resources, starts the executors (Standalone, YARN, Kubernetes)	local mode: simulated inside one JVM (`master local[4]`)
Executors	JVM processes on the worker nodes that run the tasks, keep cached partitions in memory and report back	4 task slots

3. How can Breeze be used for statistical computations on the dataset?

Breeze is a numerical library for **one JVM**: dense vectors and matrices, `mean`, `stddev`, `variance`, `dot`, element-wise operators (`/:/`), linear algebra. It is not distributed. The sensible split is: **Spark reduces the 5 million rows** to something small (aggregates, a sample, sufficient statistics), and **Breeze does the local numerical work** on that result. Collecting all 5 million rows into the driver just to use Breeze would defeat the purpose. In the code, three vectors (attendance, CGPA, placed 0/1) give mean CGPA 7.57, standard deviation 1.48, the correlations attendance~CGPA 0.952, CGPA~placed 0.955, attendance~placed 0.935, and z-scores of CGPA.

4. Propose methods to identify factors influencing placements

- **Compare groups**: placed students average CGPA 8.8 and attendance 90, not placed 6.35 and 70 (`groupBy("Placed")`).
- **Correlation** of each factor with the outcome (Breeze above, or `df.stat.corr` in Spark).
- **Model-based**: Spark MLlib `LogisticRegression` (coefficients: attendance 0.087, CGPA 0.764 on standardised features, both push towards "placed"), or `RandomForestClassifier` feature importances; `ChiSqSelector` for categorical factors.
- At 5 million rows: train/test split, cross-validation, handle class imbalance, and watch for leakage (a column that is only known after the placement).
- **Association is not cause**: CGPA and attendance may both reflect motivation or the campus. Four rows are a demonstration, not evidence.

5. Justify why Spark is preferred over traditional data-processing systems

	Traditional (single-machine tools, RDBMS, MapReduce)	Spark
Scale	limited by one machine's memory and disk (vertical scaling)	add machines (horizontal scaling)
Speed	MapReduce writes every intermediate result to disk	keeps intermediate data in memory and plans the whole DAG
Variety of work	separate systems for SQL, streaming, ML, graphs	one engine: Spark SQL, Streaming, MLlib, GraphX
Failures	re-run by hand or restart the job	recompute lost partitions from the lineage automatically
Programming	low-level or tool specific	Scala, Java, Python, R and SQL APIs

Honest limit: for data that fits one machine, plain pandas or a database is simpler and faster than starting a cluster.

Code, run on the dataset of the question (Spark + Breeze)

Code: code/sa_m4_q1.scala

```scala
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using dep org.apache.spark::spark-mllib:3.5.1
//> using dep org.scalanlp::breeze:2.1.0
//> using javaOpt <13 --add-opens options for Java 17, see the page 'How to run the programs'>
// Sample answer M4-Q1: placement analytics with Spark (distributed work) and Breeze (local numerics)
import breeze.linalg.DenseVector
import breeze.stats.{mean, stddev}
import org.apache.spark.ml.classification.LogisticRegression
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

object PlacementAnalytics {
  // Pearson correlation written with Breeze vector operations
  def pearson(a: DenseVector[Double], b: DenseVector[Double]): Double = {
    val ac = a - mean(a)
    val bc = b - mean(b)
    (ac dot bc) / math.sqrt((ac dot ac) * (bc dot bc))
  }

  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("placements").master("local[4]")
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress",
          "127.0.0.1").config("spark.driver.host", "127.0.0.1")
      .config("spark.sql.shuffle.partitions", "4").getOrCreate()
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._

    println("--- 1. architecture: the same code scales from 4 rows to 5 million rows ---")
    println("driver master: " + sc.master + " (a cluster manager URL such as yarn or k8s in production)")
    val big = spark.range(5000000L).select(col("id").as("StudentID"), (rand(1) * 40 + 60).as("Attendance"), (rand(2)
         * 5 + 5).as("CGPA"))
    println("5,000,000 synthetic students split into " + big.rdd.getNumPartitions + " partitions, one task per
        partition")
    val summary = big.agg(count("*"), round(avg("Attendance"), 1), round(avg("CGPA"), 1), round(corr("Attendance",
        "CGPA"), 3)).first()
    println("count, mean attendance, mean CGPA, correlation (independent random columns, so about 0): " +
        summary.toSeq.mkString(", "))
    println("physical plan of the aggregation (note the partial and final aggregate around one shuffle):")
    println(big.agg(avg("CGPA")).queryExecution.executedPlan.toString.linesIterator.map("  " + _).mkString("\n"))

    println("\n--- 2. the four rows of the question ---")
    val students = Seq(("S101", 92, 8.5, "Yes"), ("S102", 75, 6.8, "No"), ("S103", 88, 9.1, "Yes"), ("S104", 65,
        5.9, "No"))
      .toDF("StudentID", "Attendance", "CGPA", "Placed")
    students.groupBy("Placed").agg(count("*").as("students"), avg("Attendance").as("avg_attendance"),
        avg("CGPA").as("avg_cgpa")).orderBy("Placed").show()

    println("--- 3. Breeze on the small aggregated result (single JVM, no cluster) ---")
    val rows = students.select("Attendance", "CGPA", "Placed").collect()          // only OK because the result is
        small
    val att = DenseVector(rows.map(_.getInt(0).toDouble))
    val cgpa = DenseVector(rows.map(_.getDouble(1)))
    val placed = DenseVector(rows.map(r => if (r.getString(2) == "Yes") 1.0 else 0.0))
    println(f"CGPA: mean ${mean(cgpa)}%.2f, std dev ${stddev(cgpa)}%.2f")
    println(f"correlation attendance~CGPA ${pearson(att, cgpa)}%.3f, CGPA~placed ${pearson(cgpa, placed)}%.3f,
        attendance~placed ${pearson(att, placed)}%.3f")
    println("z-scores of CGPA: " + ((cgpa - mean(cgpa)) /:/ stddev(cgpa)).toArray.map(z => f"$z%.2f").mkString(",
        "))

    println("\n--- 4. which factors influence placement? a model on the Spark side ---")
    val prepared = new VectorAssembler().setInputCols(Array("Attendance",
        "CGPA")).setOutputCol("features").transform(
      students.withColumn("label", when(col("Placed") === "Yes", 1.0).otherwise(0.0)))
    val model = new LogisticRegression().setRegParam(0.1).setStandardization(true).fit(prepared)
    println("coefficients (Attendance, CGPA): " + model.coefficients.toArray.map(c => f"$c%.3f").mkString(", ") + "
        | both positive, so both push towards 'placed'")
    model.transform(prepared).select("StudentID", "label", "prediction").show()
    spark.stop()
  }
}
```

Real output

```
--- 1. architecture: the same code scales from 4 rows to 5 million rows ---
driver master: local[4] (a cluster manager URL such as yarn or k8s in production)
5,000,000 synthetic students split into 4 partitions, one task per partition
count, mean attendance, mean CGPA, correlation (independent random columns, so about 0): 5000000, 80.0, 7.5, 0.0
physical plan of the aggregation (note the partial and final aggregate around one shuffle):
  AdaptiveSparkPlan isFinalPlan=false
  +- HashAggregate(keys=[], functions=[avg(CGPA#4)], output=[avg(CGPA)#120])
     +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=61]
        +- HashAggregate(keys=[], functions=[partial_avg(CGPA#4)], output=[sum#124, count#125L])
           +- Project [((rand(2) * 5.0) + 5.0) AS CGPA#4]
              +- Range (0, 5000000, step=1, splits=4)

--- 2. the four rows of the question ---
+------+--------+--------------+--------+
|Placed|students|avg_attendance|avg_cgpa|
+------+--------+--------------+--------+
|    No|       2|          70.0|    6.35|
|   Yes|       2|          90.0|     8.8|
+------+--------+--------------+--------+

--- 3. Breeze on the small aggregated result (single JVM, no cluster) ---
CGPA: mean 7.57, std dev 1.48
correlation attendance~CGPA 0.952, CGPA~placed 0.955, attendance~placed 0.935
z-scores of CGPA: 0.62, -0.52, 1.03, -1.13

--- 4. which factors influence placement? a model on the Spark side ---
coefficients (Attendance, CGPA): 0.087, 0.764 | both positive, so both push towards 'placed'
+---------+-----+----------+
|StudentID|label|prediction|
+---------+-----+----------+
|     S101|  1.0|       1.0|
|     S102|  0.0|       0.0|
|     S103|  1.0|       1.0|
|     S104|  0.0|       0.0|
+---------+-----+----------+
```

M4-Q2 Smart Healthcare Data Processing with Saddle and Spark

2. Smart Healthcare Data Processing with Saddle and Spark

A hospital chain maintains patient records across multiple cities.

Dataset

PatientID	Age	Blood Pressure	Diabetes
P101	45	130	Yes
P102	32	120	No
P103	58	145	Yes
P104	29	118	No

Question:

As a Data Engineer, develop a data-processing strategy using **Saddle** and **Apache Spark**.

- How can Saddle Series and Frames be used to organize the data?
- Which Spark component would distribute processing across hospital servers?
- How would you detect abnormal patient records?
- How can large-scale healthcare analytics benefit from distributed computing?
- Compare the roles of Saddle and Spark in the solution.

The question as given (Sample Questions/6278527581963359888.jpg)

Answer, point by point

1. How can Saddle Series and Frames be used to organise the data?

- A **Series** is one labelled column: the index holds the PatientID, the values the blood pressures (`Series(bp, ids)`).
- A **Frame** is a table of such columns with row labels (patients) and column labels (Age, BP, Diabetes). It is typed: all columns of a Frame share one element type, so Diabetes is stored as 1.0 / 0.0 (or kept in a separate Series[String]).
- Index alignment, filtering (`bpSeries.filter(_ >= 140)` keeps the patient labels), sorting (`sorted`), missing-value handling (`fillNA`), column arithmetic and `Vec` statistics are available. A Series has no `mean`: use `.values.mean` (128.3).
- Saddle is the "small pandas" of Scala: one machine, in memory.

2. Which Spark component would distribute processing across hospital servers?

The work is distributed by **Spark Core's scheduler**: the DataFrame / Spark SQL layer splits the data into partitions, the cluster manager provides **executors** on the worker servers, and the driver sends one task per partition to them. In the code `repartition(2, col("Diabetes"))` shows rows spread over two partitions (P101 and P103 in partition 1, P102 and P104 in partition 0), each of which would be processed on some executor of a real cluster.

3. How would you detect abnormal patient records?

- **Clinical rules** (American Heart Association blood-pressure categories, treating the single BP number as systolic): below 120 normal, 120-129 elevated, 130-139 stage 1, 140 and above stage 2. Result: P104 normal, P102 elevated, P101 stage 1, P103 stage 2.
- **Combined rule**: diabetes and BP >= 130 marks P101 and P103 as high risk. Mean BP is 137.5 with diabetes and 119.0 without.
- **Statistical rule**: z-score or IQR. Here the largest z-score is 1.57, so no record is a statistical outlier although P103 is clinically abnormal: rules and statistics find different things, use both.
- **Validity checks**: impossible values (BP 0 or 400), missing fields, duplicate patient ids. On large data add an unsupervised detector (isolation forest, k-means distance).

4. How can large-scale healthcare analytics benefit from distributed computing?

- Millions of records from many cities are processed in minutes, because partitions are handled in parallel.
- Data can be processed where it is stored (data locality), and a failed node does not lose the job (lineage recomputation).
- Model training and cohort queries scale by adding servers.
- Patient data is sensitive: anonymise or pseudonymise identifiers, control access and encrypt in transit and at rest.

5. Compare the roles of Saddle and Spark

	Saddle	Spark
Where it runs	one JVM, data in local memory	a cluster of executors, data in partitions
Data size	what fits in one machine	far larger than one machine
Strength	labelled, aligned Series / Frame manipulation, quick exploration	parallel SQL, ETL, machine learning, streaming
Typical role here	organise and inspect a hospital's summary or a sample	read, clean, aggregate and score all hospitals' records
Together	Spark produces the aggregated result, Saddle (or Breeze) analyses it locally	

Code 1: Saddle

Code: code/sa_m4_q2_saddle.scala

```
//> using scala 2.13.12
//> using dep io.github.pityka::saddle-core:4.0.0-M14
// Sample answer M4-Q2 (Saddle part): organise the patient records on one machine with Series and Frames
import org.saddle._
import org.saddle.ops.BinOps._

object HospitalSaddle {
  def main(args: Array[String]): Unit = {
    val ids = Index("P101", "P102", "P103", "P104")
    val age = Vec(45.0, 32.0, 58.0, 29.0)
    val bp = Vec(130.0, 120.0, 145.0, 118.0)
    val diabetes = Vec(1.0, 0.0, 1.0, 0.0)                              // Yes = 1, No = 0

    println("--- a Series is one labelled column, a Frame is a table of them ---")
    val bpSeries = Series(bp, ids)
    println(bpSeries)
    val frame = Frame("Age" -> Series(age, ids), "BP" -> Series(bp, ids), "Diabetes" -> Series(diabetes, ids))
    println(frame)
    println(frame.numRows + " rows x " + frame.numCols + " columns, columns: " + frame.colIx.toVec.toSeq)

    println("--- summary statistics (a Series has no mean, so go through .values) ---")
    println(f"mean BP ${bpSeries.values.mean}%.1f, max BP ${bpSeries.values.max.get}%.0f, mean age ${age.mean}%.1f")

    println("--- abnormal records ---")
    println("BP >= 140 (stage 2 hypertension range): " + bpSeries.filter(_ >= 140.0).index.toVec.toSeq)
    val highRisk = bp.find(_ >= 130.0).toSeq.intersect(diabetes.find(_ == 1.0).toSeq).toArray     // positions that
        satisfy both conditions
    println("diabetes and BP >= 130: " + ids.take(highRisk).toVec.toSeq)
    println(f"mean BP with diabetes ${bp.take(diabetes.find(_ == 1.0).toArray).mean}%.1f, without
        ${bp.take(diabetes.find(_ == 0.0).toArray).mean}%.1f")
    println("sorted by BP: " + bpSeries.sorted.index.toVec.toSeq)
  }
}
```

Real output

```
--- a Series is one labelled column, a Frame is a table of them ---
[4 x 1]
P101 -> 130.0000
P102 -> 120.0000
P103 -> 145.0000
P104 -> 118.0000

[4 x 3]
            Age       BP Diabetes
        ------- -------- --------
P101 -> 45.0000 130.0000   1.0000
P102 -> 32.0000 120.0000   0.0000
P103 -> 58.0000 145.0000   1.0000
P104 -> 29.0000 118.0000   0.0000

4 rows x 3 columns, columns: ArraySeq(Age, BP, Diabetes)
--- summary statistics (a Series has no mean, so go through .values) ---
mean BP 128.3, max BP 145, mean age 41.0
--- abnormal records ---
BP >= 140 (stage 2 hypertension range): ArraySeq(P103)
diabetes and BP >= 130: ArraySeq(P101, P103)
mean BP with diabetes 137.5, without 119.0
sorted by BP: ArraySeq(P104, P102, P101, P103)
```

Code 2: Spark

Code: code/sa_m4_q2_spark.scala

```
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using javaOpt <13 --add-opens options for Java 17, see the page 'How to run the programs'>
// Sample answer M4-Q2 (Spark part): the same patient data processed in parallel
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

object HospitalSpark {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("hospital").master("local[2]")
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress",
          "127.0.0.1").config("spark.driver.host", "127.0.0.1")
      .config("spark.sql.shuffle.partitions", "4").getOrCreate()
    spark.sparkContext.setLogLevel("ERROR")
    import spark.implicits._

    val patients = Seq(("P101", 45, 130, "Yes"), ("P102", 32, 120, "No"), ("P103", 58, 145, "Yes"), ("P104", 29,
        118, "No"))
      .toDF("PatientID", "Age", "BloodPressure", "Diabetes")

    println("--- the DataFrame is split into partitions; each executor processes its partitions in parallel ---")
    val spread = patients.repartition(2, col("Diabetes"))                    // rows with the same key travel to the
         same partition
    spread.withColumn("partition", spark_partition_id()).orderBy("PatientID").show()

    println("--- abnormal records: rules and a statistical check ---")
    val flagged = patients
      .withColumn("bp_stage", when(col("BloodPressure") >= 140, "stage 2").when(col("BloodPressure") >= 130, "stage
          1")
        .when(col("BloodPressure") >= 120, "elevated").otherwise("normal"))
      .withColumn("high_risk", col("Diabetes") === "Yes" && col("BloodPressure") >= 130)
    flagged.orderBy("PatientID").show()
    val stats = patients.agg(avg("BloodPressure").as("mean"), stddev_pop("BloodPressure").as("sd")).first()
    val (mu, sd) = (stats.getDouble(0), stats.getDouble(1))
    println(f"z-score of each BP (mean $mu%.1f, sd $sd%.1f): " +
        patients.orderBy("PatientID").select("BloodPressure").as[Int].collect().map(v => f"${(v - mu) /
        sd}%.2f").mkString(", "))

    println("--- the SQL view of the same question ---")
    patients.createOrReplaceTempView("patients")
    spark.sql("SELECT Diabetes, COUNT(*) AS patients, ROUND(AVG(BloodPressure), 1) AS avg_bp FROM patients GROUP BY
        Diabetes ORDER BY Diabetes").show()
    spark.stop()
  }
}
```

Real output

```
--- the DataFrame is split into partitions; each executor processes its partitions in parallel ---
+---------+---+-------------+--------+---------+
|PatientID|Age|BloodPressure|Diabetes|partition|
+---------+---+-------------+--------+---------+
|     P101| 45|          130|     Yes|        1|
|     P102| 32|          120|      No|        0|
|     P103| 58|          145|     Yes|        1|
|     P104| 29|          118|      No|        0|
+---------+---+-------------+--------+---------+

--- abnormal records: rules and a statistical check ---
+---------+---+-------------+--------+--------+---------+
|PatientID|Age|BloodPressure|Diabetes|bp_stage|high_risk|
+---------+---+-------------+--------+--------+---------+
|     P101| 45|          130|     Yes| stage 1|     true|
|     P102| 32|          120|      No|elevated|    false|
|     P103| 58|          145|     Yes| stage 2|     true|
|     P104| 29|          118|      No|  normal|    false|
+---------+---+-------------+--------+--------+---------+

z-score of each BP (mean 128.3, sd 10.7): 0.16, -0.77, 1.57, -0.96
--- the SQL view of the same question ---
+--------+--------+------+
|Diabetes|patients|avg_bp|
+--------+--------+------+
|      No|       2| 119.0|
|     Yes|       2| 137.5|
+--------+--------+------+
```

M4-Q3 E-Commerce Recommendation System with Exception Handling and Spark

3. E-Commerce Recommendation System with Exception Handling and Spark

An online shopping company collects customer transactions.

Dataset

CustomerID	Product	Amount
C101	Laptop	65000
C102	Mobile	25000
C103	Laptop	70000
C104	Tablet	18000

Question:

Design a recommendation engine using Spark.

- Describe how Spark can process millions of transactions in parallel.
- Explain potential exceptions that may occur during data loading and processing.
- Propose an exception-handling strategy for missing or corrupted transaction records.
- Explain how Spark's fault-tolerance mechanism improves reliability.
- Suggest business insights that can be derived from the processed data.

The question as given (Sample Questions/6278527581963359900.jpg)

Answer, point by point

1. How can Spark process millions of transactions in parallel?

The transaction file is read as partitions (for a file in HDFS or S3 roughly one per block). Each partition becomes one task; tasks run in parallel on the executors. Parsing, filtering and the partial aggregation (`groupBy("Product")`) happen inside the tasks, and only the small partial results are shuffled to compute the totals. Adding executors adds parallelism without changing the code.

2. What exceptions may occur during loading and processing?

Stage	Problem	Example in the code
Loading	missing or unreadable input	`AnalysisException` with error class `PATH_NOT_FOUND`
Loading	corrupted or malformed lines	`C105,Phone,abc` (text in a number column), `C106,Laptop` (a column missing)
Processing	bad values	`NumberFormatException`, null customer id, negative amount
Processing	skewed or oversized data	one product with most of the rows, `OutOfMemoryError`
Cluster	task, executor, node or shuffle-fetch failure	simulated in section 4 of the code

3. Propose an exception-handling strategy for missing or corrupted records

- **Read with an explicit schema** and choose the parse mode: `PERMISSIVE` (default; bad rows become nulls and, with a `_corrupt_record` column, the raw line is kept), `DROPMALFORMED` (silently drops) or `FAILFAST` (stops the job at the first bad line). The code uses `PERMISSIVE` with the corrupt-record column and puts the two unparsable lines in a **dead-letter list** for later inspection; a third row fails a business rule (no customer id).
- **Validate** business rules after parsing (customer id present, amount greater than 0). 4 clean rows remain.
- **At record level** wrap the parsing in `Try` and count failures with an **accumulator** (3 bad records skipped), so one bad line never kills the job.
- **Fail fast where it should**: missing input path or schema errors stop the job with a clear message; but if the share of bad records is above a threshold, fail the job too, because the data source is broken.
- Catch exceptions at the driver around the action, log, alert, and `spark.stop()` in `finally`.

Watch out. `count()` can hide bad data. It needs no column values, so Spark prunes the columns and never parses the bad fields: the code shows `count()` returning all 7 lines while `collect()` (which parses everything) applies the parse mode.

4. How does Spark's fault-tolerance mechanism improve reliability?

- **Lineage**: Spark remembers how every partition was computed from the input. If a task or executor fails, only the lost partitions are recomputed.
- **Task retries**: a failed task is re-run (default up to 4 attempts, `spark.task.maxFailures`). Demonstrated: one task throws on its first attempt, Spark runs it again, and the job result is correct (360.0).
- Also: stage re-execution after a lost shuffle output, optional speculative execution of slow tasks, **checkpointing** to cut a very long lineage (and for streaming state), restart of the driver in cluster mode, replicated storage such as HDFS.

5. Suggest business insights from the processed data

- **Laptops** bring Rs. 135,000 of the Rs. 178,000 revenue of the clean rows (76%) from 2 of 4 buyers; mobiles 25,000 and tablets 18,000.
- **Next-best-product recommendation**: for every customer, take the products they have not bought and rank them by popularity (buyers, then revenue). Result: C101 and C103 (laptop buyers) get Mobile, C102 and C104 get Laptop.
- Next steps with more data: collaborative filtering with `ALS` from Spark MLlib, co-purchase rules ("people who bought a laptop also bought ..."), segmentation by spend.
- Data quality is itself an insight: 3 of 7 lines (43%) were damaged, so the source system needs fixing.

Code, run on the dataset of the question

Code: code/sa_m4_q3.scala

```scala
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using javaOpt <13 --add-opens options for Java 17, see the page 'How to run the programs'>
// Sample answer M4-Q3: a recommendation pipeline in Spark with exception handling and fault tolerance
import java.nio.file.{Files, Paths}
import java.util.concurrent.atomic.AtomicInteger

import scala.util.{Failure, Success, Try}

import org.apache.spark.{SparkException, TaskContext}
import org.apache.spark.scheduler.{SparkListener, SparkListenerTaskEnd}
import org.apache.spark.sql.{AnalysisException, SparkSession}
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types._

object Recommender {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("recommender").master("local[2,3]")      // 2 threads; a task may
        fail 3 times before the job fails
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress",
          "127.0.0.1").config("spark.driver.host", "127.0.0.1")
      .config("spark.sql.shuffle.partitions", "4").getOrCreate()
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._

    val lines = Seq("CustomerID,Product,Amount", "C101,Laptop,65000", "C102,Mobile,25000", "C103,Laptop,70000",
        "C104,Tablet,18000",
                    "C105,Phone,abc", ",Tablet,100", "C106,Laptop")                          // the last three lines
                        are damaged
    Files.write(Paths.get("transactions.csv"), lines.mkString("\n").getBytes("UTF-8"))

    println("--- 1. loading: a missing input is an exception at once ---")
    try spark.read.option("header", "true").csv("no_such_file.csv")
    catch { case e: AnalysisException => println("AnalysisException, error class " + e.getErrorClass) }

    println("\n--- 2. corrupted records: three parse modes ---")
    val schema = StructType(Seq(StructField("CustomerID", StringType), StructField("Product", StringType),
        StructField("Amount", DoubleType)))
    def read(mode: String, s: StructType = schema) = spark.read.option("header", "true").option("mode",
        mode).schema(s).csv("transactions.csv")
    println("count() only needs the row count, so Spark prunes the columns and never parses the bad values: " +
        read("DROPMALFORMED").count() + " rows")
    println("collect() parses everything: DROPMALFORMED keeps " + read("DROPMALFORMED").collect().length + " rows,
        PERMISSIVE keeps " + read("PERMISSIVE").collect().length + " rows")
    try read("FAILFAST").collect()
    catch { case e: SparkException => println("FAILFAST stops the job: " + e.getClass.getSimpleName) }

    val parsed = read("PERMISSIVE", StructType(schema.fields :+ StructField("_corrupt_record", StringType))).cache()
    val corrupt = parsed.filter(col("_corrupt_record").isNotNull)
    val good = parsed.filter(col("_corrupt_record").isNull && col("CustomerID").isNotNull && col("Amount") >
        0).drop("_corrupt_record")
    println("dead-letter lines (unparsable): " + corrupt.select("_corrupt_record").as[String].collect().toList)
    println("rows failing a business rule (no customer id): " + parsed.filter(col("_corrupt_record").isNull &&
        col("CustomerID").isNull).count())
    println("clean rows used for the model: " + good.count())

    println("\n--- 3. the same cleaning at RDD level with Try and an accumulator ---")
    val badCount = sc.longAccumulator("bad lines")
    val cleaned = sc.textFile("transactions.csv").filter(!_.startsWith("CustomerID")).flatMap { line =>
      Try { val Array(id, product, amount) = line.split(",", -1); (id, product, amount.toDouble) } match {
        case Success(t) if t._1.nonEmpty => Some(t)
        case _ => badCount.add(1); None
      }
    }
    println("good records: " + cleaned.count() + ", bad records skipped: " + badCount.value)

    println("\n--- 4. fault tolerance: one task fails on its first attempt, Spark re-runs it ---")
    val failedTasks = new AtomicInteger(0)
    sc.addSparkListener(new SparkListener {
      override def onTaskEnd(end: SparkListenerTaskEnd): Unit = if (end.taskInfo.failed)
          failedTasks.incrementAndGet()
    })
    val total = sc.parallelize(1 to 8, 4).map { x =>
      val ctx = TaskContext.get()
      if (ctx.partitionId() == 1 && ctx.attemptNumber() == 0) throw new RuntimeException("simulated executor
          failure")
      x * 10
    }.sum()
    Thread.sleep(1000)                                                                       // let the listener
        thread catch up
    println("result " + total + " (expected 360.0) after " + failedTasks.get() + " failed task attempt, re-run
        automatically from the lineage")
```

```
    println("\n--- 5. business insight and a first recommender ---")
    val popularity = good.groupBy("Product").agg(count("*").as("buyers"), sum("Amount").as("revenue"))
    popularity.orderBy(desc("revenue")).show()
    val candidates = good.select("CustomerID").distinct().crossJoin(popularity)
      .join(good.select("CustomerID", "Product"), Seq("CustomerID", "Product"), "left_anti")          // products
          the customer has not bought yet
    val rank = Window.partitionBy("CustomerID").orderBy(col("buyers").desc, col("revenue").desc)
    candidates.withColumn("rank", row_number().over(rank)).filter(col("rank") === 1)
      .select(col("CustomerID"), col("Product").as("recommended")).orderBy("CustomerID").show()
    spark.stop()
  }
}
```

Real output

```
--- 1. loading: a missing input is an exception at once ---
AnalysisException, error class PATH_NOT_FOUND

--- 2. corrupted records: three parse modes ---
count() only needs the row count, so Spark prunes the columns and never parses the bad values: 7 rows
collect() parses everything: DROPMALFORMED keeps 5 rows, PERMISSIVE keeps 7 rows
FAILFAST stops the job: SparkException
dead-letter lines (unparsable): List(C105,Phone,abc, C106,Laptop)
rows failing a business rule (no customer id): 1
clean rows used for the model: 4

--- 3. the same cleaning at RDD level with Try and an accumulator ---
good records: 4, bad records skipped: 3

--- 4. fault tolerance: one task fails on its first attempt, Spark re-runs it ---
result 360.0 (expected 360.0) after 1 failed task attempt, re-run automatically from the lineage

--- 5. business insight and a first recommender ---
+-------+------+--------+
|Product|buyers| revenue|
+-------+------+--------+
| Laptop|     2|135000.0|
| Mobile|     1| 25000.0|
| Tablet|     1| 18000.0|
+-------+------+--------+

+----------+-----------+
|CustomerID|recommended|
+----------+-----------+
|      C101|     Mobile|
|      C102|     Laptop|
|      C103|     Mobile|
|      C104|     Laptop|
+----------+-----------+
```

M4-Q4 Social Media Sentiment Analysis using Spark Big Data Architecture

4. Social Media Sentiment Analysis using Spark Big Data Architecture

A social media company wants to analyze millions of user posts generated daily.

Dataset

UserID	Post Type	Likes
U101	Technology	520
U102	Sports	310
U103	Education	750
U104	Technology	640

Question:

Design a big-data analytics architecture using Apache Spark.

- Explain how Spark's Driver Program and Executors coordinate processing.
- Which Spark components are responsible for task scheduling and execution?
- How would Spark manage increasing volumes of social media data?
- Identify possible system failures and explain how exception handling can improve robustness.
- Recommend metrics that can help identify trending topics.

The question as given (Sample Questions/6278527581963359907.jpg)

Answer, point by point

1. How do Spark's Driver Program and Executors coordinate processing?

Step	Who	What happens
1	Driver	your program creates the SparkSession / SparkContext and builds the logical plan from the transformations
2	DAG scheduler (in the driver)	cuts the plan into **stages** at every shuffle (wide transformation)
3	Task scheduler + cluster manager	asks for executors, then sends one **task** per partition of a stage to them
4	Executors	run the tasks on their partitions, keep cached blocks, send heartbeats and task results back
5	Driver	collects the results (`collect`, `show`, `write`), the next stage starts when the previous one is complete

The code asks the scheduler for its own report: one aggregation job was split into 2 stages (map side with 2 tasks, reduce side with 4 tasks), and the RDD version of the same aggregation shows the `ShuffledRDD` that marks the stage boundary.

2. Which Spark components are responsible for task scheduling and execution?

- **DAGScheduler** (stages), **TaskScheduler** with its scheduler backend (tasks to executors), the **cluster manager** (resources: Standalone, YARN, Kubernetes) and the **executors** (execution, caching through the block manager).
- At component level (the five Spark libraries): Spark Core does the scheduling and execution; Spark SQL, Structured Streaming, MLlib and GraphX are built on top of it.

3. How would Spark manage increasing volumes of social-media data?

- **Scale out**: more executors; dynamic allocation adds and removes them with the load.
- **Partitioning** by key and a sensible partition count. The demo aggregates 2 million rows with 1 partition and with 4 partitions and gets the same 4 groups: only the parallelism changes, not the code.

- **Efficient formats** (Parquet, columnar, compressed), caching of reused data, **broadcast joins** for small tables, salting to handle skewed keys.
- **Ingest continuously** with Structured Streaming (Kafka or files) instead of nightly batches.

4. Identify possible system failures and explain how exception handling improves robustness

- Failures: input file missing, malformed records, executor or node loss, network or shuffle timeouts, out-of-memory, driver failure.
- In the code a missing input file (`posts_day1.csv`) is caught as an `AnalysisException` with error class `PATH_NOT_FOUND`; the job reports it and continues with the other sources. Unparsable like counts ("n/a" and an empty string) are skipped and counted with an accumulator (2 skipped, valid counts 520, 310, 750).
- Task failures are retried by Spark itself; a long lineage is shortened with checkpoints; alerts fire when the bad-record ratio is too high.

5. Recommend metrics that help identify trending topics

Metric	Meaning	On this data
Total and average likes per post type	which topics get most reactions	Technology 1,160 (2 posts, 580 average), Education 750, Sports 310
Mentions per time window	growth of a hashtag hour by hour	#scala 0 to 2, #spark 1 to 2, #cricket 1 to 0 (invented posts)
Engagement rate	likes + comments + shares divided by impressions	needs impressions
Velocity / spike score	z-score of today's count against its history	needs history
Sentiment mix	share of positive, neutral, negative posts per topic	3 positive, 2 negative (lexicon rule, invented posts)

Note. The table of the question has no text, so the hashtag and sentiment parts use five invented posts to show the technique. A real sentiment system would use a trained model (Tokenizer, TF-IDF and logistic regression in Spark ML) instead of a word list.

Code, run on the dataset of the question

Code: code/sa_m4_q4.scala

```scala
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using javaOpt <13 --add-opens options for Java 17, see the page 'How to run the programs'>
// Sample answer M4-Q4: social-media analytics: how the driver, DAG scheduler and executors work, robustness,
    trending metrics
import scala.collection.mutable.ListBuffer
import scala.util.{Failure, Success, Try}

import org.apache.spark.scheduler.{SparkListener, SparkListenerStageCompleted}
import org.apache.spark.sql.{AnalysisException, DataFrame, SparkSession}
import org.apache.spark.sql.functions._

case class Post(UserID: String, PostType: String, Likes: Int)

object SocialMedia {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("social").master("local[2]")
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress",
          "127.0.0.1").config("spark.driver.host", "127.0.0.1")
      .config("spark.sql.shuffle.partitions", "4").config("spark.sql.adaptive.enabled", "false")
      .config("spark.sql.session.timeZone", "UTC").getOrCreate()
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._

    val stages = ListBuffer[String]()                                                    // what the scheduler
        reports
    sc.addSparkListener(new SparkListener {
      override def onStageCompleted(e: SparkListenerStageCompleted): Unit = stages.synchronized { stages +=
          s"${e.stageInfo.numTasks} tasks" }
    })

    println("--- 1. the driver plans, the executors run the tasks ---")
    val posts = Seq(Post("U101", "Technology", 520), Post("U102", "Sports", 310), Post("U103", "Education", 750),
        Post("U104", "Technology", 640)).toDF()
    val byType = posts.groupBy("PostType").agg(count("*").as("posts"), sum("Likes").as("total_likes"),
        round(avg("Likes"), 1).as("avg_likes"), max("Likes").as("top_post"))
    stages.clear()
    byType.collect()                                                                     // one action = one job
    Thread.sleep(500)
    println("the scheduler split that job into " + stages.size + " stages, tasks per stage: " + stages.mkString(",
        ") + " (map side, then reduce side)")
    byType.orderBy(desc("total_likes")).show()
    println("the same aggregation as an RDD: the ShuffledRDD marks the stage boundary")
    println(posts.rdd.map(r => (r.getString(1), r.getInt(2))).reduceByKey(_ + _).toDebugString.linesIterator.map("
        " + _).mkString("\n"))

    println("\n--- 2. growth in volume: more partitions, more parallel tasks, no code change ---")
    val big = spark.range(2000000L).select((col("id") % 4).as("bucket"), (rand(7) * 1000).cast("int").as("Likes"))
    for (parts <- Seq(1, 4)) println(parts + " partition(s): " +
        big.repartition(parts).groupBy("bucket").agg(sum("Likes")).orderBy("bucket").collect().length + " groups
        from " + big.count() + " rows")

    println("\n--- 3. failures and exception handling ---")
    def load(path: String): Option[DataFrame] = Try(spark.read.option("header", "true").csv(path)) match {
      case Success(df) => Some(df)
      case Failure(e: AnalysisException) => println("input " + path + " missing (" + e.getErrorClass + "): skip this
          source, keep the job alive"); None
      case Failure(e) => throw e
    }
    println("load result: " + load("posts_day1.csv").map(_.count()))
    val badLikes = sc.longAccumulator("unparsable likes")
    val likes = sc.parallelize(Seq("520", "310", "n/a", "750", "")).flatMap(s => Try(s.toInt).toOption.orElse {
        badLikes.add(1); None })
    println("valid like counts: " + likes.collect().toList + ", skipped: " + badLikes.value)

    println("\n--- 4. trending topics from (invented) timestamped posts ---")
    val feed = Seq(("U101", "2026-11-10 09:05:00", "Loving the new #spark release, great speed"),
                   ("U102", "2026-11-10 09:40:00", "Terrible match, bad refereeing #cricket"),
                   ("U103", "2026-11-10 10:10:00", "Great course on #spark and #scala"),
                   ("U104", "2026-11-10 10:30:00", "#spark job failed again, awful error"),
                   ("U101", "2026-11-10 10:50:00", "Happy with #scala great docs")).toDF("UserID", "time", "text")
    val tags = feed.withColumn("ts", to_timestamp(col("time")))
      .withColumn("tag", explode(expr("filter(split(lower(text), ' '), w -> w like '#%')")))
      .groupBy(window(col("ts"), "1 hour").getField("start").as("hour"), col("tag")).count()
    tags.orderBy("hour", "tag").show(false)
    val positive = Set("loving", "great", "happy", "good")
    val negative = Set("terrible", "bad", "awful", "failed")
    val sentiment = udf((t: String) => { val w = t.toLowerCase.split("\\W+"); w.count(positive) - w.count(negative)
        })
    feed.withColumn("score", sentiment(col("text"))).withColumn("label", when(col("score") > 0,
        "positive").when(col("score") < 0, "negative").otherwise("neutral"))
      .groupBy("label").count().orderBy("label").show()
    spark.stop()
  }
```

```
}
```

Real output

```
--- 1. the driver plans, the executors run the tasks ---
the scheduler split that job into 2 stages, tasks per stage: 2 tasks, 4 tasks (map side, then reduce side)
+----------+-----+-----------+---------+--------+
|  PostType|posts|total_likes|avg_likes|top_post|
+----------+-----+-----------+---------+--------+
|Technology|    2|       1160|    580.0|     640|
| Education|    1|        750|    750.0|     750|
|    Sports|    1|        310|    310.0|     310|
+----------+-----+-----------+---------+--------+

the same aggregation as an RDD: the ShuffledRDD marks the stage boundary
  (2) ShuffledRDD[24] at reduceByKey at sa_m4_q4.scala:38 []
   +-(2) MapPartitionsRDD[23] at map at sa_m4_q4.scala:38 []
      |  MapPartitionsRDD[22] at rdd at sa_m4_q4.scala:38 []
      |  SQLExecutionRDD[21] at rdd at sa_m4_q4.scala:38 []
      |  MapPartitionsRDD[20] at rdd at sa_m4_q4.scala:38 []
      |  MapPartitionsRDD[19] at rdd at sa_m4_q4.scala:38 []
      |  ParallelCollectionRDD[18] at rdd at sa_m4_q4.scala:38 []

--- 2. growth in volume: more partitions, more parallel tasks, no code change ---
1 partition(s): 4 groups from 2000000 rows
4 partition(s): 4 groups from 2000000 rows

--- 3. failures and exception handling ---
input posts_day1.csv missing (PATH_NOT_FOUND): skip this source, keep the job alive
load result: None
valid like counts: List(520, 310, 750), skipped: 2

--- 4. trending topics from (invented) timestamped posts ---
+-------------------+--------+-----+
|hour               |tag     |count|
+-------------------+--------+-----+
|2026-11-10 09:00:00|#cricket|1    |
|2026-11-10 09:00:00|#spark  |1    |
|2026-11-10 10:00:00|#scala  |2    |
|2026-11-10 10:00:00|#spark  |2    |
+-------------------+--------+-----+

+--------+-----+
|   label|count|
+--------+-----+
|negative|    2|
|positive|    3|
+--------+-----+
```

M4-Q5 Financial Risk Analysis using Breeze, Saddle and Spark

5. Financial Risk Analysis using Breeze, Saddle, and Spark

A banking organization analyzes customer loan information.

Dataset

CustomerID	Income (₹)	Loan Amount (₹)	Loan Status
L101	650000	300000	Approved
L102	350000	400000	Rejected
L103	800000	500000	Approved
L104	250000	350000	Rejected

Question:

Develop an analytical framework using **Breeze**, **Saddle**, and **Apache Spark**.

- How can Saddle be used to structure and preprocess the data?
- Which Breeze functions could help compute statistical measures and risk indicators?
- How can Spark distribute loan-analysis tasks across a cluster?

The question as given (Sample Questions/6278527581963359908.jpg)

Answer, point by point

> **Note.** The screenshot ends after the third bullet; if the original question has more bullets they could not be read. The three visible bullets are answered.

1. How can Saddle be used to structure and preprocess the data?

- Put the numbers in a labelled **Frame** (`Income`, `Loan` per CustomerID); keep the text column `Status` in a separate `Vec[String]` because a Frame holds one element type.
- **Preprocess**: fill a missing income (`fillNA(_ => mean)` turns the NaN into 566,667 in the demo), derive new columns with element-wise arithmetic (`loan / income` gives the loan-to-income ratio), filter by condition, sort (riskiest first: L104, L102, L103, L101).
- **Group statistics** with index positions: mean ratio 0.543 for approved loans and 1.271 for rejected loans.

2. Which Breeze functions help compute statistical measures and risk indicators?

Need	Breeze	Result
Centre and spread	`mean`, `stddev`, `variance` on a `DenseVector`	income: mean 512,500, standard deviation 256,174
Loan-to-income ratio	`ln /:/ inc` (element-wise division)	0.462, 1.143, 0.625, 1.400
Income left after the loan	`inc - ln` (vector subtraction)	350,000; -50,000; 300,000; -100,000
Correlation, covariance	`dot` on centred vectors	correlation income~loan 0.429
Standardised income	`(inc - mean(inc)) /:/ stddev(inc)`	0.54, -0.63, 1.12, -1.02
Rule-based flag	comparison on the ratio	ratio above 1.0 means the loan exceeds the income: L102 and L104 (both rejected)

The ratio separates the two groups cleanly in this sample (approved: 0.46 and 0.63; rejected: 1.14 and 1.40), which is why it is a useful risk indicator. Four loans cannot prove a rule; in practice add the debt-to-income ratio, credit history and employment data.

3. How can Spark distribute the loan-analysis tasks across a cluster?

- Load the loan table as a DataFrame (from HDFS, S3 or a database) and **repartition** by a suitable key (customer id range, branch, region). Each partition is one task: the code shows L101 and L104 in partition 0, L102 and L103 in partition 1.

- Row-wise work (the ratio, the risk band) runs inside the tasks on all partitions at once; aggregations (`groupBy("Status")`) first aggregate per partition and then combine, so little data is shuffled.
- A small shared value such as the risk threshold is sent once to every executor with a **broadcast variable**.
- Model training with **MLlib** `LogisticRegression`: the coefficient of the loan-to-income ratio is -2.536, the coefficient of income is about 0 once the ratio is in the model. A heavier loan lowers the chance of approval.
- The driver then gets only the small result (summary table, coefficients), possibly for Saddle or Breeze to analyse further.

Code 1: Saddle and Breeze

Code: code/sa_m4_q5_saddle_breeze.scala

```
//> using scala 2.13.12
//> using dep io.github.pityka::saddle-core:4.0.0-M14
//> using dep org.scalanlp::breeze:2.1.0
// Sample answer M4-Q5 (Saddle and Breeze part): structure and preprocess the loan data, compute statistics and risk
     indicators
import org.saddle._
import org.saddle.ops.BinOps._
import breeze.linalg.DenseVector
import breeze.stats.{mean, stddev, variance}

object LoanRisk {
  def main(args: Array[String]): Unit = {
    val ids = Index("L101", "L102", "L103", "L104")
    val income = Vec(650000.0, 350000.0, 800000.0, 250000.0)
    val loan = Vec(300000.0, 400000.0, 500000.0, 350000.0)
    val status = Vec("Approved", "Rejected", "Approved", "Rejected")

    println("--- Saddle: labelled, typed structures and preprocessing ---")
    val numbers = Frame("Income" -> Series(income, ids), "Loan" -> Series(loan, ids))
    println(numbers)
    val gap = Vec(650000.0, Double.NaN, 800000.0, 250000.0)                    // an income that was not recorded
    println("missing income filled with the mean: " + gap.fillNA(_ => gap.mean).toSeq)
    val ratio = loan / income                                                   // element-wise division gives a
        new risk column
    val ratioSeries = Series(ratio, ids)
    println(ratioSeries.mapValues(r => f"$r%.3f"))
    val approved = status.find(_ == "Approved").toArray
    val rejected = status.find(_ == "Rejected").toArray
    println(f"mean loan-to-income: approved ${ratio.take(approved).mean}%.3f, rejected
        ${ratio.take(rejected).mean}%.3f")
    println("riskiest first: " + ratioSeries.sorted.reversed.index.toVec.toSeq)

    println("--- Breeze: statistics and risk indicators (numerical work on dense vectors) ---")
    val inc = DenseVector(income.toArray)
    val ln = DenseVector(loan.toArray)
    println(f"income: mean ${mean(inc)}%.0f, std dev ${stddev(inc)}%.0f, variance ${variance(inc)}%.3e")
    val lti = ln /:/ inc                                                        // loan-to-income ratio
    val surplus = inc - ln                                                      // income left after the loan
        amount
    println("loan-to-income : " + lti.toArray.map(v => f"$v%.3f").mkString(", "))
    println("income - loan  : " + surplus.toArray.map(_.toLong).mkString(", "))
    val ic = inc - mean(inc)
    val lc = ln - mean(ln)
    println(f"correlation income~loan ${(ic dot lc) / math.sqrt((ic dot ic) * (lc dot lc))}%.3f, covariance ${(ic
        dot lc) / (inc.length - 1)}%.3e")
    val zIncome = (inc - mean(inc)) /:/ stddev(inc)
    println("z-scores of income: " + zIncome.toArray.map(z => f"$z%.2f").mkString(", "))
    println("high-risk flag (loan above income): " + lti.toArray.map(_ > 1.0).mkString(", "))
  }
}
```

Real output

```
--- Saddle: labelled, typed structures and preprocessing ---
[4 x 2]
             Income        Loan
         ----------- -----------
L101 -> 650000.0000 300000.0000
L102 -> 350000.0000 400000.0000
L103 -> 800000.0000 500000.0000
L104 -> 250000.0000 350000.0000

missing income filled with the mean: ArraySeq(650000.0, 566666.6666666666, 800000.0, 250000.0)
[4 x 1]
L101 -> 0.462
L102 -> 1.143
L103 -> 0.625
L104 -> 1.400

mean loan-to-income: approved 0.543, rejected 1.271
riskiest first: ArraySeq(L104, L102, L103, L101)
--- Breeze: statistics and risk indicators (numerical work on dense vectors) ---
income: mean 512500, std dev 256174, variance 6.563e+10
loan-to-income : 0.462, 1.143, 0.625, 1.400
income - loan  : 350000, -50000, 300000, -100000
correlation income~loan 0.429, covariance 9.375e+09
z-scores of income: 0.54, -0.63, 1.12, -1.02
high-risk flag (loan above income): false, true, false, true
```

Code 2: Spark

Code: code/sa_m4_q5_spark.scala

```scala
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using dep org.apache.spark::spark-mllib:3.5.1
//> using javaOpt <13 --add-opens options for Java 17, see the page 'How to run the programs'>
// Sample answer M4-Q5 (Spark part): distribute the loan analysis over a cluster (here: local threads)
import org.apache.spark.ml.classification.LogisticRegression
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

object LoanSpark {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("loans").master("local[2]")
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress",
          "127.0.0.1").config("spark.driver.host", "127.0.0.1")
      .config("spark.sql.shuffle.partitions", "4").getOrCreate()
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._

    val loans = Seq(("L101", 650000.0, 300000.0, "Approved"), ("L102", 350000.0, 400000.0, "Rejected"),
                    ("L103", 800000.0, 500000.0, "Approved"), ("L104", 250000.0, 350000.0,
                        "Rejected")).toDF("CustomerID", "Income", "Loan", "Status")

    println("--- 1. a derived risk column, computed in parallel on every partition ---")
    val scored = loans.withColumn("LoanToIncome", col("Loan") / col("Income"))
      .withColumn("risk", when(col("LoanToIncome") > 1.0, "high").when(col("LoanToIncome") > 0.6,
          "medium").otherwise("low"))
    scored.withColumn("LoanToIncome", round(col("LoanToIncome"), 3)).orderBy("CustomerID").show()          //
        rounded for display only
    scored.groupBy("Status").agg(count("*").as("loans"), round(avg("LoanToIncome"), 3).as("avg_ratio"),
      round(min("LoanToIncome"), 3).as("min"), round(max("LoanToIncome"), 3).as("max")).orderBy("Status").show()

    println("--- 2. how the work is spread: each partition becomes one task on some executor ---")
    loans.repartition(2).rdd.mapPartitionsWithIndex((i, rows) => Iterator(i ->
        rows.map(_.getString(0)).toList.sorted)).collect().sortBy(_._1)
      .foreach { case (i, ids) => println("partition " + i + ": " + ids) }

    println("--- 3. a shared threshold sent once to every executor (broadcast variable) ---")
    val limit = sc.broadcast(1.0)
    println(loans.rdd.map(r => (r.getString(0), r.getDouble(2) / r.getDouble(1) >
        limit.value)).collect().sortBy(_._1).toList)

    println("--- 4. a model trained with Spark MLlib: what drives approval? ---")
    val data = new VectorAssembler().setInputCols(Array("Income",
        "LoanToIncome")).setOutputCol("features").transform(
      scored.withColumn("label", when(col("Status") === "Approved", 1.0).otherwise(0.0)))
    val model = new LogisticRegression().setRegParam(0.1).setStandardization(true).fit(data)
    println("coefficients (Income, LoanToIncome): " + model.coefficients.toArray.map(c => f"$c%.3f").mkString(", ")
        + " | a negative sign on the ratio: a heavier loan lowers the chance of approval")
    model.transform(data).select("CustomerID", "Status", "prediction").orderBy("CustomerID").show()
    spark.stop()
  }
}
```

Real output

```
--- 1. a derived risk column, computed in parallel on every partition ---
+----------+--------+--------+--------+------------+------+
|CustomerID|  Income|    Loan|  Status|LoanToIncome|  risk|
+----------+--------+--------+--------+------------+------+
|      L101|650000.0|300000.0|Approved|       0.462|   low|
|      L102|350000.0|400000.0|Rejected|       1.143|  high|
|      L103|800000.0|500000.0|Approved|       0.625|medium|
|      L104|250000.0|350000.0|Rejected|         1.4|  high|
+----------+--------+--------+--------+------------+------+

+--------+-----+---------+-----+-----+
|  Status|loans|avg_ratio|  min|  max|
+--------+-----+---------+-----+-----+
|Approved|    2|    0.543|0.462|0.625|
|Rejected|    2|    1.271|1.143|  1.4|
+--------+-----+---------+-----+-----+

--- 2. how the work is spread: each partition becomes one task on some executor ---
partition 0: List(L101, L104)
partition 1: List(L102, L103)
--- 3. a shared threshold sent once to every executor (broadcast variable) ---
List((L101,false), (L102,true), (L103,false), (L104,true))
--- 4. a model trained with Spark MLlib: what drives approval? ---
coefficients (Income, LoanToIncome): 0.000, -2.536 | a negative sign on the ratio: a heavier loan lowers the chance
    of approval
+----------+--------+----------+
|CustomerID|  Status|prediction|
+----------+--------+----------+
|      L101|Approved|       1.0|
|      L102|Rejected|       0.0|
|      L103|Approved|       1.0|
|      L104|Rejected|       0.0|
+----------+--------+----------+
```

Notes and limits

What was run, and what was not

- 18 questions, 25 runs on this computer: 21 programs (9 Python, 5 Scala, 7 on Spark, Breeze or Saddle), 3 files that are meant to fail (to capture compiler messages) and 1 REPL session. The outputs in the document are the captured outputs.
- Versions: Python 3.10, pandas 2.3.3, NumPy 1.26.4, SciPy 1.15.3, scikit-learn 1.7.2, Matplotlib 3.10.0, Seaborn 0.13.2, Plotly 7.1.0, Bokeh 3.9.2, Folium 0.20.0, Beautiful Soup 4.15.0, Scrapy 2.19.0, NLTK 3.10.4, ChatterBot 1.2.15 (with spaCy 3.8.18); Scala 2.13.12 on Java 17, Spark 3.5.1 (local mode), Breeze 2.1.0, Saddle 4.0.0-M14 (a maintained fork; the original Saddle does not exist for Scala 2.13).
- The datasets are exactly the ones printed in the screenshots. Invented material: the five posts of M4-Q4, the map coordinates of M2-Q5, the HTML pages of M2-Q3 (stand-ins for the two websites; no website was contacted), and the weather service of M1-Q2 (a small local server that fails on purpose; no weather provider was contacted).
- Two screenshots are cut off at the bottom (M4-Q1 and M4-Q5). Only the visible bullets are answered.
- M1-Q1 consists of five screenshots, and the one that would hold the opening scenario text (file number ...034) is not in the folder. That answer follows the datasets and the seven tasks.
- M1-Q3 answers the question with a keyword matcher built on NLTK plus a ChatterBot layer; it is a small demonstration with a handful of FAQs, not a trained support bot.
- Spark ran on one machine, so statements about clusters (executors on many nodes, cluster managers) come from the Spark documentation and from the structure of the program, not from a cluster run. Spark writes to disk and Structured Streaming were not used (they need extra setup on Windows).
- With only four or five rows per question, statistics and machine-learning results are demonstrations of method, not findings.